

Langages de la JVM 1

1ère année du Cycle Ingénieur en Informatique

Aurélien Max

Année 2025-26

université
PARIS-SACLAY



POLYTECH[®]
PARIS-SACLAY



Introduction au module

Apprendre Java en 2025 ?

Dans les notes de cours

- ( 8) : indique qu'une fonctionnalité est disponible (parfois en *preview feature*) à partir d'une version particulière de Java
 - ↪ : indique qu'une notion sera reprise plus loin
 - des références et sources (spécifications, articles, livres, vidéos) peuvent être suivies *via* des liens cliquables
-

📅 Situation de Java dans l'écosystème (2025)

- langage indéniablement important
- mais en perte de vitesse ?

❓ où se situe Java relativement aux autres langages (importants) de programmation

- est-il cantonné à certains usages ?
- n'est-il là que pour du code *legacy* ?

❓ où se situe Java relativement aux autres langages (importants) de la machine virtuelle Java (JVM)

Java (~1995) ➡ Groovy (~2003) ➡ Scala (~2004) ➡ Clojure (~2007) ➡ Kotlin (~2011)

🔗 conditions du maintien du langage (depuis 1995) >

☰ éléments importants

- compatibilité des nouvelles VMs et du code nouveau avec le code *legacy*
- évolution du langage pour favoriser le travail des développeurs
- maintenance et amélioration du langage (sécurité, performance)
- poursuite des développements de nouvelles applications & bibliothèques
- interopérabilité avec d'autres langages
- ...

🔗 choix du langage >

Java est un bon langage d'étude

- il est ancien, maintient une bonne compatibilité, et est toujours utilisé
- représentant accessible de la programmation orientée objet, ayant inclus des éléments de programmation fonctionnelle
- il fonctionne dans une machine virtuelle (VM) qui évolue et bénéficie d'un écosystème important

Nous intéressent également :

- le fait qu'il a subi des évolutions qui illustrent l'évolution des technologies

- le fait que d'autres langages inspirés par lui sont depuis apparus (et peuvent être facilement interopérables en s'exécutant dans la même VM)
-

② Ne manque-t-il pas un aspect difficile à ignorer ?



🔗 ... >

L'évolution du positionnement des outils d'IA (LLM) dans les tâches de développement

Java comme langage d'étude

Support pour l'étude de la programmation orientée objet

- langage généralement reconnu comme "accessible"
- d'autres langages importants étudiés (notamment C++)
- (langage très enseigné pour la POO)

Langage aujourd'hui très répandu

- très demandé en milieu industriel (nouveaux projets, évolutions, maintenances)
 - très bien outillé (ex: IDEs [Eclipse](#), [IntelliJ IDEA](#), [Visual Studio Code](#))
 - bonnes performances et portabilité
 - bibliothèques standard (APIs) très importantes
 - nombreuses bibliothèques additionnelles (Open Source et propriétaires)
 - réutilisation des **machines virtuelles** (JVM) par d'autres langages (ex. Scala, Groovy, Kotlin)
-

Littérature abondante, relativement bien traduite

- descriptions génériques et spécifiques
- nombreuses références aux formats papier et électronique

Documentation standard efficace

Exemple

Documentation des API Java 25

OVERVIEW

CLASS

USE

TREE

PREVIEW

NEW

DEPRECATED

INDEX

SEARCH

HELP

Java SE 25 & JDK 25

java.base > java.lang > **IO**

Search documentation (type /)

Contents

Filter contents

Description

Method Summary

Method Details

println(Object)

println()

print(Object)

readln()

readln(String)

Class IO

[java.lang.Object](#)
[java.lang.IO](#)

public final class **IO**

extends [Object](#)

A collection of static methods that provide convenient access to [System.in](#) and [System.out](#) for line-oriented input and output.

The [readln\(\)](#) and [readln\(String\)](#) methods decode bytes read from [System.in](#) into characters. The charset used for decoding is specified by the [stdin.encoding](#) property. If this property is not present, or if the charset it names cannot be loaded, then UTF-8 is used instead. Decoding always replaces malformed and unmappable byte sequences with the charset's default replacement string.

Charset decoding is set up upon the first call to one of the [readln](#) methods. Decoding may buffer additional bytes beyond those that have been decoded to characters returned to the application. After the first call to one of the [readln](#) methods, any subsequent use of [System.in](#) results in unspecified behavior.

API Note:

The expected use case is that certain applications will use only the [readln](#) methods to read from the standard input, and they will not mix these calls with other techniques for reading from [System.in](#).

Since:

25

Method Summary

All Methods

Static Methods

Concrete Methods

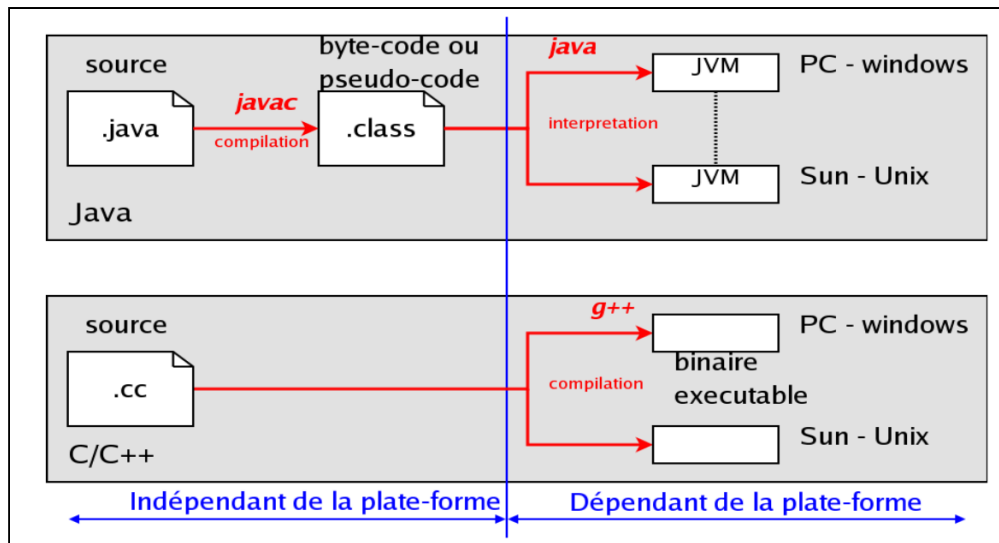
Modifier and Type	Method	Description
static void	print(Object obj)	Writes a string representation of the specified object to the standard output.
static void	println()	Writes a line separator to the standard output.

Aspects importants du langage Java

Aspects souvent mis en avant

- **Programmation orientée objet**
 - se concentrer sur les objets et leurs interfaces
 - **Simplicité**
 - notamment relativement à C++
 - **Fiabilité et sécurité**
 - ex. pas d'accès direct à la mémoire
 - **Architecture neutre et portabilité**
 - ex. spécifications des tailles des types de données primitifs
 - **Large gamme d'applications possibles**
 - ex. applications métiers, web (*back-end*), mobile, embarqué
 - **Très large diffusion**
 - ex. relativement facile de trouver des développeurs Java
-

Portabilité : *write once, run anywhere?*



- Utilisation de **machines virtuelles Java** (*Java Virtual Machines* (JVM)) ☞, réutilisées par d'autres langages (ex. Scala, Groovy, Kotlin)
 - Amélioration sensible des temps d'exécution avec (notamment) l'introduction de **compilation à la volée** (*just-in-time*) et des progrès sur la **récupération de ressources** (*garbage collection*)
-

Évolution du langage Java

Release Family	GA	End Of Support Life (EOSL)
25 LTS	16th September 2025	September 2033
24	18th March 2025	September 2025
23	17th September 2024	March 2025
22	19th March 2024	September 2024
21 LTS	19th September 2023	September 2031
20	21st March 2023	September 2023
19	20th September 2022	March 2023
18	22nd March 2022	September 2022
17 LTS	14th September 2021	September 2029
16	16th March 2021	September 2021
15	15th September 2020	March 2021
14	17th March 2020	September 2020
13	17th September 2019	March 2020
12	19th March 2019	September 2019
11 LTS	25th September 2018	September 2026
10	20th March 2018	September 2018
9	21st September 2017	March 2018
8 LTS	18th March 2014	December 2030
7 LTS	11th July 2011	July 2022
6 LTS	12th December 2006	December 2018
5 LTS	30th September 2004	July 2015
4 LTS	13th February 2002	March 2013
3 LTS	8th May 2000	April 2011
2 LTS	4th December 1998	December 2003
1.1 LTS	28th March 1997	January 2003
1.0 LTS	23rd January 1996	October 2002

[source](#)

② compatibilité ascendante : que du positif ?

☞ "As the platform has matured, yet continued to evolve, many community members have naturally come to expect that their investments in Java-based systems (...) will be preserved. Any program running on a previous release of the platform must also run -unchanged- on an implementation of [a newer Java platform]. (There are [rare] exceptions to this general rule [(...) which] typically involve serious issues such as security.)"

Processus d'évolution du langage

L'évolution de Java :

- est dirigée par le **Java Community Process** (JCP)
- est alimentée par des **Java Enhancement Proposals** (JEP)
- repose sur des **Java Specification Requests** (JSR)
- repose sur la **Java Language Specification** (JLS) (JSR 90118)
- permet l'intégration de **preview features**

≡ Exemples

JEP 445: Unnamed Classes and Instance Main Methods (Preview)



- Installing
- Contributing
- Sponsoring
- Developers' Guide
- Vulnerabilities
- JDK GA/EA Builds
- Mailing lists
- Wiki · IRC
- Bylaws · Census
- Legal
- Workshop**
- JEP Process**
- Source code**
- Mercurial
- GitHub
- Tools**
- Git
- jTreg harness
- Groups**
- (overview)
- Adoption
- Build
- Client Libraries
- Compatibility & Specification Review
- Compiler
- Conformance
- Core Libraries
- Governing Board
- HotSpot
- IDE Tooling & Support

JEP 445: Unnamed Classes and Instance Main Methods (Preview)

<i>Author</i>	Ron Pressler
<i>Owner</i>	Jim Laskey
<i>Type</i>	Feature
<i>Scope</i>	SE
<i>Status</i>	Closed / Delivered
<i>Release</i>	21
<i>Component</i>	specification / language
<i>Discussion</i>	amber dash dev at openjdk dot org
<i>Effort</i>	S
<i>Reviewed by</i>	Alex Buckley, Brian Goetz
<i>Endorsed by</i>	Brian Goetz
<i>Created</i>	2023/02/13 13:58
<i>Updated</i>	2023/08/16 16:34
<i>Issue</i>	8302326

Summary

Evolve the Java language so that students can write their first programs without needing to understand language features designed for large programs. Far from using a separate dialect of Java, students can write streamlined declarations for single-class programs and then seamlessly expand their programs to use more advanced features as their skills grow. This is a **preview language feature**.

JEP 512: Compact Source Files and Instance Main Methods



Installing
Contributing
Sponsoring
Developers' Guide
Vulnerabilities
JDK GA/EA Builds

Mailing lists
Wiki · IRC
Mastodon
Bluesky

Bylaws · Census
Legal
Workshop
JEP Process
Source code
GitHub
Mercurial

Tools
Git
JRebel harness

Groups
(overview)
Adoption
Build
Client Libraries
Compatibility &
Specification
Review
Compiler
Conformance
Core Libraries
Governing Board
HotSpot
IDE Tooling & Support
Internationalization
JMX
Members
Networking
Porters

JEP 512: Compact Source Files and Instance Main Methods

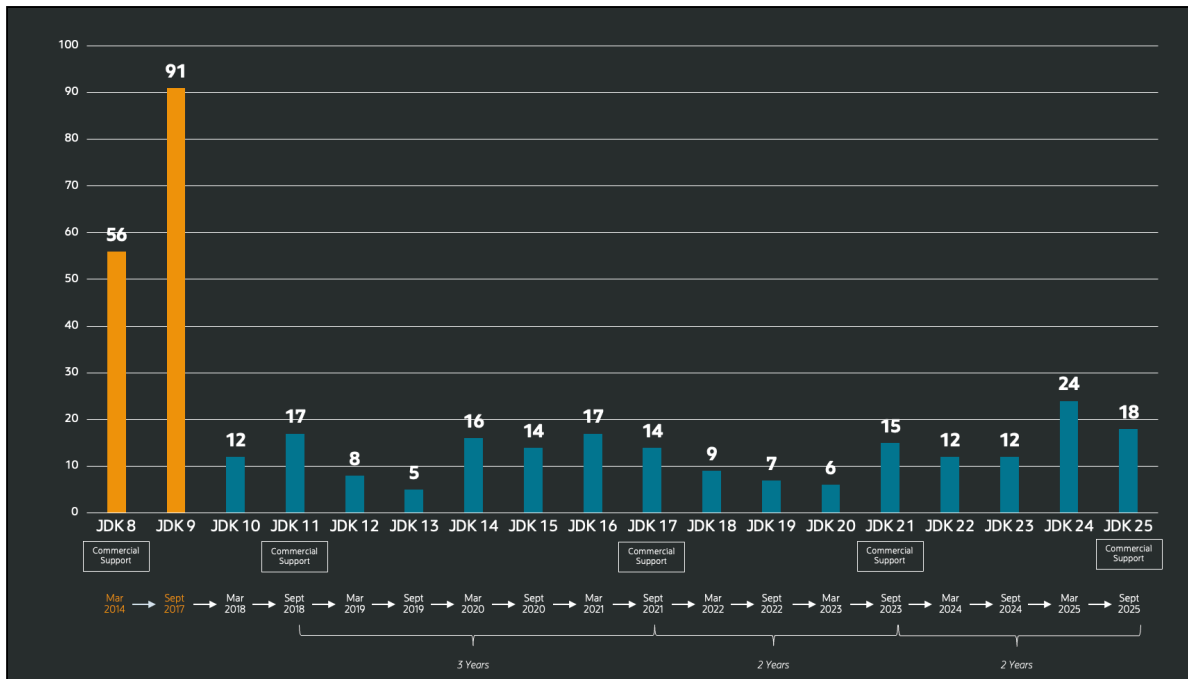
<i>Authors</i>	Ron Pressler, Jim Laskey, & Gavin Bierman
<i>Owner</i>	Gavin Bierman
<i>Type</i>	Feature
<i>Scope</i>	SE
<i>Status</i>	Closed / Delivered
<i>Release</i>	25
<i>Component</i>	specification / language
<i>Discussion</i>	amber dash dev at openjdk dot org
<i>Relates to</i>	JEP 511: Module Import Declarations JEP 495: Simple Source Files and Instance Main Methods (Fourth Preview)
<i>Reviewed by</i>	Alex Buckley, Brian Goetz
<i>Endorsed by</i>	Brian Goetz
<i>Created</i>	2024/11/21 11:58
<i>Updated</i>	2025/07/11 06:45
<i>Issue</i>	8344699

Summary

Evolve the Java programming language so that beginners can write their first programs without needing to understand language features designed for large programs. Far from using a separate dialect of the language, beginners can write streamlined declarations for single-class programs and then seamlessly expand their programs to use more advanced features as their skills grow. Experienced developers can likewise enjoy writing small programs succinctly, without the need for constructs intended for programming in the large.

Apports des évolutions du langage

- Les évolutions rythmées visent à faire de Java un "*langage moderne, compétitif et sécurisé*"
- Nombre de JEP par version de Java ([source](#))



- Les évolutions introduites ne sont pas toujours majeures, la mise à niveau vers les versions LTS présente toutefois des bénéfices

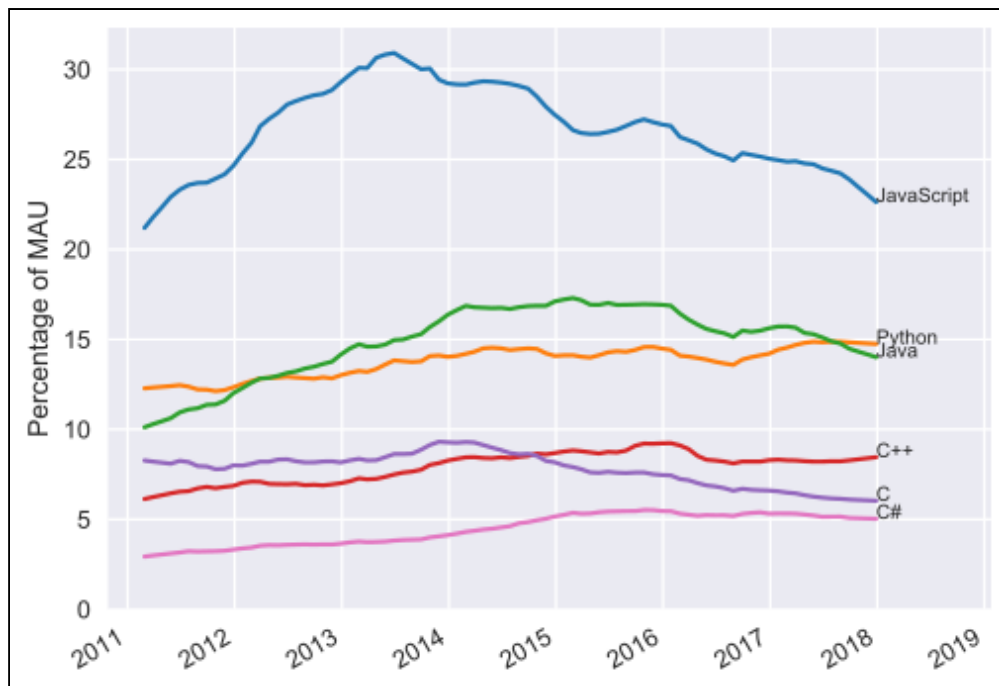
🔗 "When Java 17 was released in 2021 as a 'long term support' version, I wrote an article dissecting its features and came to the conclusion that it had a few nice features, but none that were compelling reasons to upgrade. Except one: tens of thousands of bug fixes." -- [Cay Horstmann](#)

Adoption du langage

② Comment peut-on estimer l'adoption d'un langage de programmation ?

GitHub : mesure du nombre de contributeurs actifs par langage

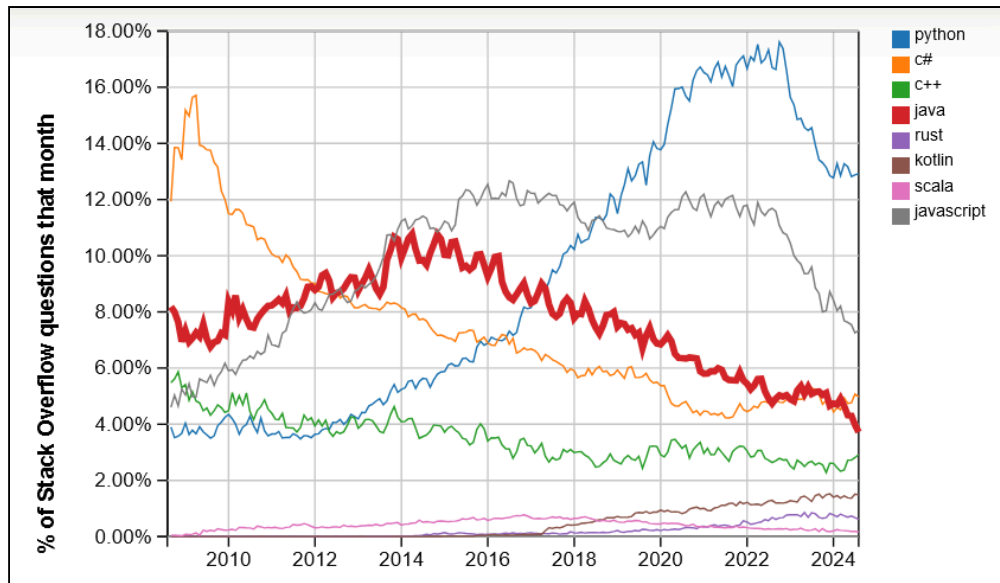
([source](#))



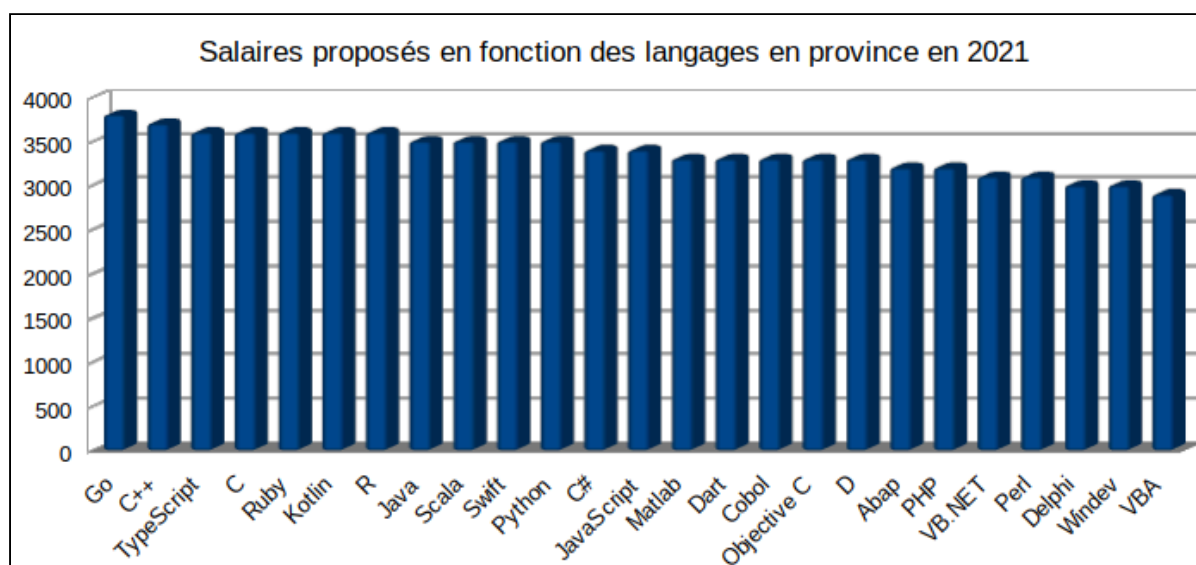
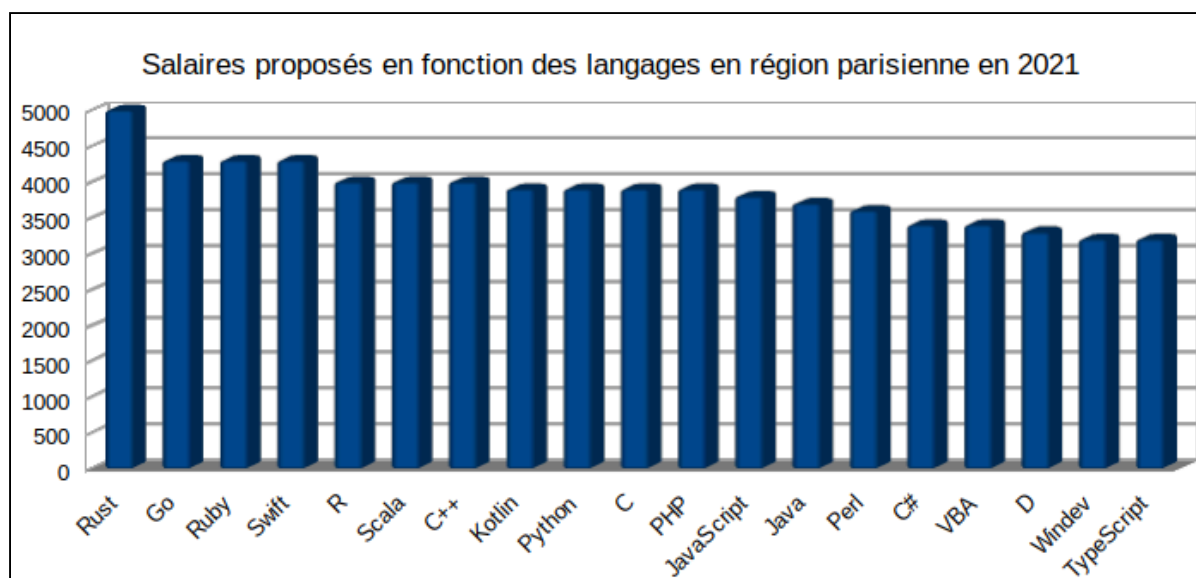
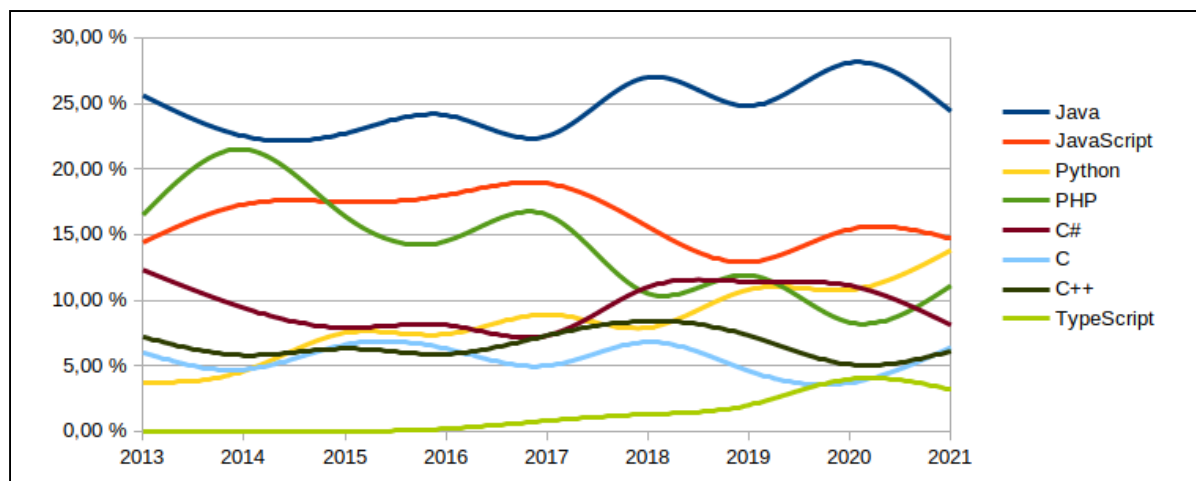
Indice TIOBE : intégration automatique du nombre de développeurs, éditeurs et cours en ligne ([source](#))

Oct 2025	Oct 2024	Change	Programming Language		Ratings	Change
1	1			Python	24.45%	+2.55%
2	4	▲		C	9.29%	+0.91%
3	2	▼		C++	8.84%	-2.77%
4	3	▼		Java	8.35%	-2.15%
5	5			C#	6.94%	+1.32%
6	6			JavaScript	3.41%	-0.13%
7	7			Visual Basic	3.22%	+0.87%
8	8			Go	1.92%	-0.10%
9	10	▲		Delphi/Object Pascal	1.86%	+0.19%
10	11	▲		SQL	1.77%	+0.13%

Stack Overflow Trends : proportion de questions posées concernant chaque langage ([source](#))



Offres d'emplois postées sur le portail Emploi de Developpez.com ([source](#))



Critères de performance des langages

Étude scientifique étudiant des langages de programmation selon :

- le temps de calcul
- la mémoire utilisée
- l'énergie consommée
- des combinaisons de ces critères

[Source](#)

Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. 2017. Energy Efficiency across Programming Languages: How Do Energy, Time, and Memory Relate? In Proceedings of 2017 ACM SIGPLAN International Conference on Software Language Engineering (SLE'17)

	Energy		Time		Mb
(c) C	1.00	(c) C	1.00	(c) Pascal	1.00
(c) Rust	1.03	(c) Rust	1.04	(c) Go	1.05
(c) C++	1.34	(c) C++	1.56	(c) C	1.17
(c) Ada	1.70	(c) Ada	1.85	(c) Fortran	1.24
(v) Java	1.98	(v) Java	1.89	(c) C++	1.34
(c) Pascal	2.14	(c) Chapel	2.14	(c) Ada	1.47
(c) Chapel	2.18	(c) Go	2.83	(c) Rust	1.54
(v) Lisp	2.27	(c) Pascal	3.02	(v) Lisp	1.92
(c) Ocaml	2.40	(c) Ocaml	3.09	(c) Haskell	2.45
(c) Fortran	2.52	(v) C#	3.14	(i) PHP	2.57
(c) Swift	2.79	(v) Lisp	3.40	(c) Swift	2.71
(c) Haskell	3.10	(c) Haskell	3.55	(i) Python	2.80
(v) C#	3.14	(c) Swift	4.20	(c) Ocaml	2.82
(c) Go	3.23	(c) Fortran	4.20	(v) C#	2.85
(i) Dart	3.83	(v) F#	6.30	(i) Hack	3.34
(v) F#	4.13	(i) JavaScript	6.52	(v) Racket	3.52
(i) JavaScript	4.45	(i) Dart	6.67	(i) Ruby	3.97
(v) Racket	7.91	(v) Racket	11.27	(c) Chapel	4.00
(i) TypeScript	21.50	(i) Hack	26.99	(v) F#	4.25
(i) Hack	24.02	(i) PHP	27.64	(i) JavaScript	4.59
(i) PHP	29.30	(v) Erlang	36.71	(i) TypeScript	4.69
(v) Erlang	42.23	(i) Jruby	43.44	(v) Java	6.01
(i) Lua	45.98	(i) TypeScript	46.20	(i) Perl	6.62
(i) Jruby	46.54	(i) Ruby	59.34	(i) Lua	6.72
(i) Ruby	69.91	(i) Perl	65.79	(v) Erlang	7.20
(i) Python	75.88	(i) Python	71.90	(i) Dart	8.64
(i) Perl	79.58	(i) Lua	82.91	(i) Jruby	19.84

Time & Memory	Energy & Time	Energy & Memory	Energy
C • Pascal • Go	C	C • Pascal	
Rust • C++ • Fortran	Rust	Rust • C++ • Fortran • Go	
Ada	C++	Ada	
Java • Chapel • Lisp • Ocaml	Ada	Java • Chapel • Lisp	Java
Haskell • C#	Java	OCaml • Swift • Haskell	
Swift • PHP	Pascal • Chapel	C# • PHP	Dart •
F# • Racket • Hack • Python	Lisp • Ocaml • Go	Dart • F# • Racket • Hack • Python	Jav
JavaScript • Ruby	Fortran • Haskell • C#	JavaScript • Ruby	
Dart • TypeScript • Erlang	Swift	TypeScript	
JRuby • Perl	Dart • F#	Erlang • Lua • Perl	
Lua	JavaScript	JRuby	
	Racket		
	TypeScript • Hack		
	PHP		
	Erlang		
	Lua • JRuby		
	Ruby		

Introduction au module

Questions abordées dans le module

- éléments de base de la programmation en Java
- héritage et polymorphisme
- prévention et gestion des erreurs
- programmation générique et collections
- introduction à la JVM

Approche du module >

- éléments de cours
- pratique : TPs et projet
- présentations techniques (liste de sujets proposés)

Évaluation du module >

- travaux pratiques (individuel) : 30 %
 - projet (individuel) : 30 %
 - examen écrit (individuel) : 40 %
-

Exemple introductif

parlons de ce qu'on connaît bien : des étudiants

≡ On doit développer une application de gestion qui manipulera (au moins initialement) des étudiants de Polytech

- en simplifiant (beaucoup) ceux-ci sont caractérisés par leur *prénom, nom, et numéro d'étudiant*
- on s'intéresse en particulier au fait qu'ils aient déjà fait leur *inscription administrative* ou non
- donc, pour un étudiant, on devrait au moins pouvoir interroger sur son statut d'inscription, le modifier, et obtenir une représentation sous forme de chaîne de caractères pour affichage

② Exemple: solution fondée sur le langage C ? >

② Et si notre programme devait être plus ambitieux ?

- d'autres entités (ex. **enseignants, personnels administratifs**), d'autres types d'**étudiants** dont le règlement des études peut être (partiellement) différent ?
- et si d'autres devaient le faire évoluer ou l'utiliser comme partie d'une application plus complexe?

- En **programmation orientée objet**, on va faire de l'**encapsulation** ⇨
- Définition d'un "moule formel" (**classe** ⇨) ayant vocation à être instancié pour des données concrètes (**objets** ⇨)
- Mécanisme de définition de la **visibilité** ⇨ des éléments

```
1  /* une classe représentant un étudiant Polytech */
2  public class EtudiantPolytech {
3
4      // des données membres de la classe
5      // des comportements (fonctions) membres
6      // une documentation intégrée
7      // ...
8
9  }
```

- On encapsule la déclaration des données membres (par objet)
- Ces déclarations sont caractérisées (au moins) par un **identificateur** (ex. `estInscrit`), un **type** (ex. `boolean`), une **visibilité** (ex. `private`)

```
1  /* une classe représentant un étudiant Polytech */
2  public class EtudiantPolytech {
3
4      /* des attributs : données privées ici */
5      private String prénom, nom;
6      private int identifiant;
7      private boolean estInscrit;
8
9      // ...
10 }
```

- Notion de fonctions membres : **méthodes** ➤
- Des **méthodes d'instance** ➤ pourront être appelées sur les objets de la classe
 - elles permettront par exemple d'accéder en consultation à des données de l'objet

```
1  /* une classe représentant un étudiant Polytech */
2  public class EtudiantPolytech {
3
4      // ...
5
6      /* une méthode publique pour obtenir le nom complet
       */
7      public String getNomCompleet() {
8          return prénom + " " + nom + "(inscription : " + \
9              (estInscrit ? "oui" : "non") + ")";
10     }
11
12     // ...
13
14 }
```

- Les **méthodes d'instance** permettent également de modifier les données d'un objet
- Elles implémentent les contrôles requis et la logique de modification

```
1  /* une classe représentant un étudiant Polytech */
2  public class EtudiantPolytech {
3
4      /* une méthode publique pour inscrire l'étudiant */
5      public boolean inscrire() {
6          if (estInscrit) {
7              System.err.println("Cet étudiant est déjà
inscrit");
8              return false ;
9          }
10         estInscrit = true ;
11         return true;
12     }
13
14     // ...
15 }
```

- Un type de méthode particulier permet de créer des objets de la classe : les **constructeurs** ➞

```
1  /* une classe représentant un étudiant Polytech */
2  public class EtudiantPolytech {
3
4      /* des attributs de la classe : données privées */
5      private String prénom, nom;
6      private int identifiant;
7      private boolean estInscrit;
8
9      /* un constructeur d'instances */
10     public EtudiantPolytech(String lePrénom, String
        leNom, int id) {
11         prénom = lePrénom; nom = leNom;
12         identifiant = id;
13         estInscrit = false;
14     }
15
16     // ...
17 }
```

```
1  /* une classe représentant un étudiant Polytech */
2  public class EtudiantPolytech {
3      /* des attributs de la classe : données privées */
4      private String prénom, nom;
5      private int identifiant;
6      private boolean estInscrit;
7
8      /* un constructeur d'instances */
9      public EtudiantPolytech(String lePrénom, String
leNom, int id) {
10         prénom = lePrénom; nom = leNom;
11         identifiant = id;
12         estInscrit = false;
13     }
14
15     /* une méthode publique pour obtenir le nom complet
*/
16     public String getNomComplet() {
17         return prénom + " " + nom + "(inscription : " + \
18             (estInscrit ? "oui" : "non") + ")";
19     }
20
21     /* une méthode publique pour inscrire l'étudiant */
22     public boolean inscrire() {
23         if (estInscrit) {
24             System.err.println("Cet étudiant est déjà
inscrit");
25             return false ;
26         }
27         estInscrit = true ;
28         return true;
29     }
30 }
```

- En l'état, la classe `EtudiantPolytech` ne définit pas un programme
- Il faut un point d'entrée et une séquence d'instructions : cela se fait au moyen d'une **méthode de classe** ➤ particulière, éventuellement dans une autre classe: `main`
- A minima, nous pouvons instancier une classe et manipuler l'objet obtenu



programme de test

```
1 public class TestEtudiant {
2
3     public static void main(String[] args) {
4         /* instantiation d'un objet de la classe
           EtudiantPolytech */
5         EtudiantPolytech unEtudiant = new
           EtudiantPolytech("Joseph", \
6             "Fourier", 1234);
7         /* série d'appels de méthodes de la classe
           EtudiantPolytech */
8         System.out.println("Nom : " +
           unEtudiant.getNomCompleet());
9         unEtudiant.inscrire();
10        System.out.println("Nom : " +
           unEtudiant.getNomCompleet());
11        unEtudiant.inscrire();
12    }
13 }
```



compilation

```
$ javac -classpath . TestEtudiant.java
```



exécution

```
1 $ java -classpath . TestEtudiant
```

- 2 Nom : Joseph Fourier (inscription: non)
 - 3 Nom : Joseph Fourier (inscription: oui)
 - 4 Cet étudiant est déjà inscrit
-

🔗 Limites de l'approche

- Un étudiant a tout de même des caractéristiques communes avec d'autres entités de notre programme: on voudrait autant que possible établir des relations entre classes et tirer d'autres bénéfices
- Possibilité de définir une classe `Personne`, qui impliquera des changements pour notre classe `EtudiantPolytech`

```
1  /* une classe représentant une personne */
2  public class Personne {
3      private String nom;
4      private String prénom;
5
6      public Personne(String lePrénom, String leNom) {
7          nom = leNom;
8          prénom = lePrénom;
9      }
10
11     public String getNomComplet() {
12         return prénom + " " + nom;
13     }
14 }
```

- Un `étudiant` étant une *personne* comme les autres, on établit un lien entre la classe `Personne` et la classe `EtudiantPolytech`
 - on parle d'**héritage**  



La classe EtudiantPolytech devient une sous-classe de la classe Personne

```
1  /* une classe représentant un étudiant Polytech
2     comme une "spécialisation" d'une Personne */
3
4  public class EtudiantPolytech extends Personne {
5
6      // ...
7
8  }
```

- Un objet de type `EtudiantPolytech` a en outre des données qui lui sont propres
 - ces données n'étaient pas pertinentes au niveau de la classe parente `Personne`

```
1  /* une classe représentant un étudiant Polytech
2     comme une "spécialisation" d'une Personne */
3
4  public class EtudiantPolytech extends Personne {
5
6      private int identifiant;
7      private boolean estInscrit;
8
9      // ...
10
11 }
```

- La classe `EtudiantPolytech` a également des méthodes qui lui sont propres

```
1  /* une classe représentant un étudiant Polytech
2     comme une "spécialisation" d'une Personne */
3
4  public class EtudiantPolytech extends Personne {
5
6      public boolean inscrire() {
7          if (estInscrit) return false ;
8          estInscrit = true ;
9          return true;
10     }
11
12     // ...
13
14 }
```

- La classe `EtudiantPolytech` peut également **redéfinir**, pour les spécialiser, des méthodes existant dans la classe `Personne`

❓ comment respecte-t-on le contrat de la définition de la classe parente ?

```
1  /* une classe représentant un étudiant Polytech
2     comme une "spécialisation" d'une Personne */
3
4  public class EtudiantPolytech extends Personne {
5
6     /* méthode commune (par défaut) à tous les
7        étudiants
8        elle redéfinit (en l'utilisant, ici) celle
9        définie dans Personne */
10    @Override
11    public String getNomCompleet() {
12        return super.getNomCompleet() + "(inscription : "
13        + \
14        (estInscrit ? "oui" : "non") + ")";
15    }
16    // ...
17 }
```

- Il nous faut enfin adapter le **constructeur** de la classe

`EtudiantPolytech`

② comment sera initialisée la partie `Personne` des futurs objets `EtudiantPolytech` ?

```
1  /* une classe représentant un étudiant Polytech
2     comme une "spécialisation" d'une Personne */
3
4  public class EtudiantPolytech extends Personne {
5
6      public EtudiantPolytech(String lePrénom, String
7          leNom, int id) {
8          /* appel du constructeur de la super-classe */
9          super(lePrénom, leNom);
10         identifiant = id;
11         estInscrit = false;
12     }
13     // ...
14 }
```

```
1  /* une classe représentant un étudiant Polytech
2     comme une "spécialisation" d'une Personne */
3
4  public class EtudiantPolytech extends Personne {
5      private int identifiant;
6      private boolean estInscrit;
7
8      public EtudiantPolytech(String lePrénom, String
leNom, int id) {
9          /* appel du constructeur de la super-classe */
10         super(lePrénom, leNom);
11         identifiant = id;
12         estInscrit = false;
13     }
14
15     /* méthode commune (par défaut) à tous les
étudiants
16         elle redéfinit (en l'utilisant, ici) celle
définie dans Personne */
17     @Override
18     public String getNomComplet() {
19         return super.getNomComplet() + "(inscription : "
+ \
20             (estInscrit ? "oui" : "non") + ")";
21     }
22
23     public boolean inscrire() {
24         if (estInscrit) return false ;
25         estInscrit = true ;
26         return true;
27     }
28 }
```

- Le principe ayant mené à la séparation de `EtudiantPolytech` et `Personne` peut également s'appliquer entre `EtudiantPolytech` et une nouvelle classe plus générale `Etudiant`



une classe représentant un étudiant comme spécialisation d'une personne

```
1  public class Etudiant extends Personne {
2      private int identifiant;
3      private boolean estInscrit;
4
5      public Etudiant(String lePrénom, String leNom, int
    id) {
6          super(lePrénom, leNom);
7          identifiant = id;
8          estInscrit = false;
9      }
10
11     @Override
12     public String getNomComplet() {
13         return super.getNomComplet() + "(inscription : "
    + \
14             (estInscrit ? "oui" : "non") + ")";
15     }
16
17     public boolean inscrire() {
18         if (estInscrit) return false ;
19         estInscrit = true ;
20         return true;
21     }
22 }
```

- On peut donc tirer profit de la définition d' `Etudiant` pour définir une relation d'héritage avec `EtudiantPolytech`

❓ quelle relation existe-t-il entre `EtudiantPolytech` et `Personne` à présent ?

```
1  public class EtudiantPolytech extends Etudiant {
2      /* données spécifiques à un étudiant Polytech */
3      public static final byte MIN_POINTS_QUITUS = 10;
4      private byte pointsQuitus;
5
6      public EtudiantPolytech(String lePrénom, String
leNom, int id) {
7          super(lePrénom, leNom, id);
8          pointsQuitus = 0;
9      }
10
11     /* méthodes spécifiques à un étudiant Polytech */
12     public boolean ajouterPointsQuitus(byte points) {
13         pointsQuitus += points;
14         return quitusValidé();
15     }
16
17     public boolean quitusValidé() {
18         return pointsQuitus >=
EtudiantPolytech.MIN_POINTS_QUITUS;
19     }
20 }
```

Exemple introductif : premier bilan

- Les champs des instances ont typiquement vocation à être **privés** (**encapsulation** )
 - les **méthodes d'accès** (*setters*) permettent un contrôle efficace sur les modifications d'état des instances
 - Les **constructeurs** permettent de créer des **instances** des classes (**objets**)
 - ils doivent avoir une **visibilité publique** (**public**) pour pouvoir être utilisés depuis n'importe quelle (autre) classe
 - Un (objet) étudiant de Polytech (classe **EtudiantPolytech**) est un étudiant (**Etudiant**), et donc également une personne (**Personne**)
 - Les **champs** et les **comportements** publics d'une **Personne** sont donc **hérités** par un **Etudiant** (distinctions de **visibilité**)
 - Cela n'empêche pas notamment :
 - un **EtudiantPolytech** (classe dérivée) de **définir** de nouveaux champs (ex. `pointsQuitus`) et/ou de nouveaux comportements (ex. `ajouterPointsQuitus`)
 - un **Etudiant** (classe dérivée) de **redéfinir** un comportement d'une classe dont il a hérité (`getNomComple`)
 - Java offre un mécanisme pour faire référence aux définitions des méthodes présentes dans la chaîne d'héritage (**super**)
-

🔗 La suite

- L'approche orientée objet permet une programmation très **modulaire**, et donc de capitaliser sur des définitions plus générales
 - À la limite, possibilité d'avoir des définitions... *abstraites* ↪
-

Éléments de base de la programmation en Java

Programmes en Java

- Un programme Java met (typiquement) en jeu des **objets** » ayant un **état** » (**données**) et des **comportements** » (**méthodes**) associés
- Les objets sont des instances de **classes** », qui définissent de façon formelle comment *interagir* avec leurs objets (l'**interface** ») et de quoi ils sont constitués (l'**implémentation** »)
- Pour pouvoir être manipulés, les objets de classes doivent tout d'abord être *instanciés*
 - allocation mémoire des données, initialisation et retour d'une **référence** »
 - cela est fait (notamment) par un **constructeur** » pour la classe de l'objet

🔥 POO

- la programmation orientée objet met en avant l'interface plutôt que les données
 - les détails d'implémentation n'ont typiquement pas à être connus de l'utilisateur d'une classe
-

- Typiquement, un programme instancie des objets et utilise leur interface
- Il existe également des **méthodes de classes** (`static`) qui n'opèrent pas sur des objets
 - ces méthodes sont toutefois bien encapsulées dans des classes
 - cela permet par exemple de définir un programme avec la signature prédéfinie : `public static void main(String[] args)`

```
1  public class SuperProgramme {  
2  
3      public static void main(String[] args) {  
4          MaClasse instance = new MaClasse();  
5          instance.faitQuelqueChose();  
6          /* ... */  
7      }  
8  
9  }
```

Convention du langage

Un fichier `.java` porte le nom de son unique classe publique (ex. `SuperProgramme.java`)

✎ fichiers source compacts (☞21)

- Les fichiers source compacts ([*compact source files*](#)) ☞21(permettent l'ajout de méthodes qui appartiennent à une classes déclarée implicitement
- Une telle méthode `main` pourra donc jouer le rôle de point d'entrée pour un programme, ex. :

```
1  void main() {  
2      System.out.println("Hello, world!");  
3  }
```

- L'exécution et la compilation se font alors avec une unique commande, ex. :

 *compilation*

```
$ java MonProgramme.java
```

📖 motivations

Cet ajout au langage (intégration définitive ☞25() est principalement destiné aux apprenants et permet l'écriture de *scripts* sans définition explicite de classes.

Données en Java

🔥 Java est un langage fortement typé

- le type de chaque variable doit être déclaré
- des vérifications sont faites à la compilation

📋 Java manipule des types objets et des types primitifs

- c'est une concession importante au "tout objet"
 - cependant, les **types primitifs** ont des classes associées ↩
 - les **types objets** contiennent soit une **référence** ➡ à un objet existant, soit une absence de référence (valeur `null`)
-

Types primitifs

- Il existe 8 types primitifs, dont le domaine de valeurs *ne dépend pas de la machine sur laquelle le code s'exécute*
 - **Types entiers** (uniquement *signés*)
 - `byte` (1 octet / 8 bits) : $-128 (-2^7)$ à $127 (+2^7 - 1)$
 - `short` (2 octets / 16 bits) : $-32\,768$ à $32\,767$
 - `int` (4 octets / 32 bits) : environ +/- 2 milliards
 - `long` (8 octets / 64 bits) : environ +/- 9 milliards de milliards
 - **Types à virgule flottante** (uniquement *signés*)
 - `float` (4 octets / 32 bits) : environ +/- $3.40E+38$
 - `double` (8 octets / 64 bits) : environ +/- $1.79E+308$
 - **Type booléen**
 - `boolean` : `true` ou `false`
 - **Type caractère**
 - `char` (au moins 2 octets) : caractères Unicode (ex : `\u03C0` → π)
-

Opérateurs des types primitifs

- On trouve classiquement :
 - les opérateurs arithmétiques usuels
 - les opérateurs de {pré-,post}-{incr,decr}-mentation
(`c++`, `--d`)
 - les opérateurs relationnels et booléens (`==`, `!=`, `&&`, `||`,
`expression ? valIfTrue : valIfFalse`)
 - les opérateurs binaires (`&`, `|`, `^`, `~`, `>>`)
 - Les fonctions et les constantes mathématiques existent sous forme de méthodes et de constantes de classe de la classe des API standard `java.lang.Math`
 - ex. `Math.sin`, `Math.PI`
 - La conversion entre types primitifs peut se faire (avec possiblement une perte d'information) par **transtypage**
 - ex. `int i = (int)myFloat;`
 - pour faire un arrondi, utiliser une méthode de classe
 - ex. `int i = Math.round(d);`
 -  équivalent pour le transtypage entre objets ↷
-

Lien types primitifs et objets

- Il existe des "classes enveloppes" (*wrapper classes*) correspondant à chacun des types primitifs
 - ex. `java.lang.Double`
 - ces classes sont **inaltérables** $\Rightarrow \Leftarrow$ et **non dérivables** $\Rightarrow \Leftarrow$
 - elles sont notamment utiles lorsque des objets sont attendus (concession sur la performance)
 - elles accueillent des méthodes utiles portant sur les types concernés
 - ex : dans `java.lang.Integer`, `static int parseInt(String s)` renvoie l'entier représenté par une chaîne de caractères

Conversion entre types primitifs et classes enveloppes

- explicite :
 - `Integer integer = new Integer(3);`
 - `int i = integer.intValue();`
 - automatique (i.e. par le compilateur) :
 - un `int` est converti si un `Integer` est attendu : **autoboxing** $\Rightarrow$
 - un `Integer` est converti si un `int` est attendu : **unboxing** $\Leftarrow$
-

Déclaration de variables

- La déclaration d'une **variable** associe un type à un **identificateur** :
 - `int entier;`
 - `String chaîne;`
- Les identificateurs peuvent contenir tout caractère Unicode
 - cf. `Character.isJavaIdentifierStart` et `isJavaIdentifierPart`
 - ⚠ attention cependant à l'accessibilité du code
- Les identificateurs sont sensibles à la casse

✍ Suivi de conventions de codage

Exemple de convention fréquente : première lettre en minuscule, mots accolés avec une majuscule ("*camelCase*")

```
byte ageDuCapitaine;
```

Déclaration de constantes (`final`)

- Les **constantes** sont définies avec le mot-clé `final`
 - plus de modification possible une fois la première affectation effectuée
 - ex. définition d'une constante de classe

```
public static final float PI = 3.1415f;
```

⚠ Variables `final`

Une variable objet `final` ne peut pas être modifiée une fois affectée, mais cela ne dit rien de l'objet référencé (ne le rend pas non modifiable) ↔

✍ Montrer l'intention

On peut qualifier de `final` toute variable pour laquelle on veut clairement montrer l'intention qu'elle ne soit pas modifiée.

Initialisation des variables

- Il est possible d'initialiser une variable (par affectation) lors de sa déclaration
 - `byte ageDuCapitaine = 55;`
 - `String nomDuCapitaine = "Crocdur";`
- Les **variables locales** doivent être initialisées avant leur première utilisation

```
1 public void méthode() {  
2     int entier;  
3     int entier2 = entier;  
4 }
```

```
1 error: variable entier1 might not have been  
   initialized  
2             int entier2 = entier1;
```

Initialisation de champs

Les **champs des objets** reçoivent des valeurs par défaut qui dépendent de leur type ↪

Inférence de type (`var`) (☕ 10)

- Mécanisme d'**inférence de type** (`var`) (☕ 10) pour les variables locales initialisées lors de leur déclaration, ex.

- `var monBooléen = true;`
- `var maChaîne = "hello, world";`
- `var notesEtudiants = new HashMap<String, List<Note>>();`
-  `var monObjet = null;`

② le recours à l'inférence de type est-elle souhaitable ?

② pourquoi est-elle limitée aux variables locales initialisées lors de leur déclaration ?

Manipulation de grands nombres

- Possibilité de manipuler des nombres avec une séquence arbitraire de chiffres avec les classes `java.math.BigInteger` et `java.math.BigDecimal`
 - on obtient des instances à l'aide d'une méthode statique
ex. `BigInteger grosEntier = BigInteger.valueOf(4);`
 - les opérateurs sur ces données sont définis par le biais de méthodes d'instance

ex. `BigInteger grosEntier3 =
grosEntier1.add(grosEntier2);`

❓ serait-il approprié de pouvoir réutiliser les opérateurs arithmétiques sur ces types ? >

- cela n'est pas permis en Java (ex. `grosEntier1 + grosEntier2`)
 - certains langages orientés objet permettent de **surcharger** les opérateurs arithmétiques (ex.: C++, Kotlin)
-

Le type chaîne de caractères (`String`)

- Les chaînes de caractères (type `java.lang.String`) représentent des séquences de caractères Unicode
 - raccourcis syntaxiques commodes
 - `String c1 = new String("salut");`
 - `String c2 = "la terre";`
 - `String c3 = c1 + " " + c2; // concaténation`
 - les constantes caractères peuvent utiliser des points de code Unicode
 - `String chaîne= "Java\u2122"; // Java™`
- Blocs de texte pour chaînes littérales sans échappements (🔥 15)

```
1  public String exempleDocumentHTML() {
2      return
3          """
4          <html>
5          <body>
6          <span>Ceci est un document HTML</span>
7          </body>
8          </html>
9          """;
10 }
```

Construction de chaînes de caractères

- Les instances de type `String` sont **inaltérables** (*immutable*) ↪
 - (choix notamment motivé par des raisons de sécurité et la possibilité de partager efficacement les représentations en mémoire des chaînes)
 - pour obtenir l'altération d'une chaîne, il faut passer par une nouvelle instance

```
String c2 = c1.substring(0, 3);
```

- mais il est également possible de redéfinir/recycler une variable référence

```
c1 = c1.substring(0, 3);
```

- La classe `java.lang.StringBuilder` permet de construire progressivement des chaînes de façon efficace

```
1  StringBuilder sb = new StringBuilder();
2  sb.append(e2, "!");
3  // ...
4  String s = sb.toString(); // récupération de la
    chaîne construite comme une instance de String
```

Les tableaux

- Représentation de **collections** ↪ pour des valeurs de même type référencées par un indice
 - `int[] tableau;` ou `int tableau[];`
 - les indices commencent à 0
 - un adressage hors des capacités du tableau mène à une erreur à l'exécution traitée par **exception** ⚠ ↪
 - La création se fait à l'aide de l'opérateur `new` (création d'objets) :
 - `int[] tableau = new int[tailleTableau];`
 - initialisation des éléments à une valeur par défaut pour le type (`null` pour des objets)
 - possibilité d'initialiser un tableau lors de sa déclaration
 - `int[] tableau = { 1, 2, 3, 4, 5};`
 - `String[] lettres = new String[]{"alpha", "beta"};`
-

- La taille d'un tableau ne peut ensuite plus être modifiée, mais on peut modifier les éléments individuels
 - le champ `length` permet de connaître le nombre d'éléments d'un tableau
 - `for(int indice = 0; indice < tableau.length; indice++)`
 - raccourci syntaxique pour parcours par itérateur sur tableau/collection (🔥5)
 - `for (int entier : tableauDEntiers)`

🔧 Méthodes utilitaires sur tableaux

La classe `java.util.Arrays` contient de nombreuses méthodes de classe utiles

- `copyOf`, `fill`, `print`, `sort`, `etc.`
-

Entrées/sorties élémentaires

- La classe `java.lang.System` contient des variables de classe de type `java.io.PrintStream` `out` (sortie standard) et `err` (sortie d'erreurs)

- utilisés pour écrire sur ces flux de sortie

```
System.err.print("Une erreur s'est produite");
```

- l'affichage peut être réalisé de façon formatée (cf. `printf` du C)

```
System.err.printf("Il y a %d erreur(s)\n",  
nbErreurs);
```

- formatage possible avec la méthode statique `format`

```
String r = String.format("Nom : %s", nom);
```

❗ ces flux standard peuvent être redirigés lors du démarrage du programme

```
$ java MaClasse > sortie.txt 2> log.txt
```

- `System` a de même une variable de classe de type `java.io.InputStream in` (entrée standard)
- Utilisation de la classe `java.util.Scanner` pour lire de façon formatée

```
1  java.util.Scanner scanner = new  
   java.util.Scanner(System.in);  
2  String chaîne = scanner.nextLine();  
3  int entier = scanner.nextInt();
```

Flux d'exécution

- Dans les grandes lignes, Java reprend et enrichit les structures de contrôle du langage C
- Définition de **blocs d'instructions** avec `{ ... }`

❗ Possibilité d'omission des accolades pour un bloc avec une seule instruction, mais déconseillée

📋 instructions conditionnelles

- `if (condition) instruction [ else instruction ]`
- `switch` (types primitifs et types énumérés, `String` (§7)) : nombreuses évolutions (expression (§14), pattern matching) ↷

📋 instructions de boucles

- `do instruction while (condition);`
- `while (condition) instruction;`
- `for` du langage C (ainsi que sur les collections ↷)

📋 instructions de rupture de séquence

- `return`, `break` et `continue` (possibilité blocs nommés), propagation d'**exceptions** (`throw` ↷)

```
1  lecture_données: // bloc nommé
2  while (...) {
3      for (...) {
4          if (...)
```

```
5         break lecture_données; // sort de la
    boucle nommée
6     }
7 }
```

Packages

- Les classes peuvent être organisées dans une hiérarchie de **packages** 📁 ("paquets")
 - association d'une classe à un package avec une instruction **package** en tête de fichier

```
package fr.polytech.ia;
```
 - recommandation fréquente : utiliser un nom de domaine à l'envers suivi d'une hiérarchie conceptuelle

🔍 Où se trouve une classe *par défaut*? >

Une classe est définie par défaut dans un *package par défaut*.

- ⚠ non recommandé excepté pour de petits tests

🔍 Peut-on mettre n'importe quelle classe dans n'importe quel package ? >

En fait, oui...

📄 les packages standard (ex. `java.`) sont désormais protégés

📄 Organisation sur disque

- Les répertoires de fichiers source (`.java`) et bytecode (`.class`) doit reprendre la structure définie dans les noms de packages
- Indication au compilateur / à la machine virtuelle de leur emplacement (*classpath*)

```
$ javac -classpath ./java/src  
fr.polytech.ia.SuperIA.java
```

```
1 /home/toto/java/src
2   \_ fr
3     \_ polytech
4       \_ ia
5         \_ SuperIA.java
```

Fichiers archives (JAR)

- Pour le **déploiement**, on préfère diffuser un unique fichier
 - assimilable grossièrement à un programme complet sans ses dépendances externes (éventuellement avec certaines de ses données) ou à une librairie
- utilisation du format **JAR** (Java Archive)
 - (hiérarchie de fichiers avec métadonnées compressée au format ZIP (compression importante du bytecode))
- une archive peut contenir plusieurs classes "exécutables" (classes publiques munies d'une méthode `main`) voire aucune (librairie)

☰ Construction de fichiers JAR avec l'outil du JDK

```
$ jar cvf ClassesIA.jar *  
$ jar cvf ClassesIA.jar *.class icon.gif # ajout de  
ressources  
$ jar cvfe JeuDEchecs.jar  
fr.polytech.ia.echecs.ClientJeuDEchec *.class # archive  
exécutable  
$ java -jar JeuDEchecs.jar # exécution
```

① automatisation forte dans les pratiques de CI/CD (*continuous integration/continuous delivery*) ↪

Importations

- Possibilité de qualifier complètement un élément (*fully qualified name*)

```
java.util.Date deadline = new java.util.Date();
```

- Pour utiliser (notamment) des classes depuis d'autres classes, on utilise l'instruction `import` avec un chemin de classe en tête de fichier (après l'instruction `package`)

- une classe en particulier : `import java.util.ArrayList;`

- un ensemble de types : `import java.net.*;`

- un champ ou une méthode **statique** ↪ : `import static`

```
java.lang.System.out;
```

- Simple commodité lexicale (pas d'inclusion effective de code comme en C) si pas d'ambiguïté sur les noms
 - permet de déclarer au lecteur d'une classe toutes ses dépendances externes

```
1  import java.util.Date;
2  // ...
3  Date deadline = new Date();
```

① Le contenu du package des API standard `java.lang` est importé automatiquement (ex. `java.lang.String`)

🔗 Les packages permettent un nouveau niveau de contrôle sur la visibilité

- les types et membres `public` sont accessibles en dehors du package
- l'appartenance à un même package définit un niveau particulier pour la visibilité : `package-private`

⚠ la visibilité `package-private` s'obtient sans mot-clé ➡

Contrôles d'accès

Modificateur	Classe	Package	Sous-classe	Autres
<code>public</code>				
<code>protected</code>				
<code>private</code>				

Recommandations

- utiliser `private` à moins d'avoir une bonne raison de ne pas le faire
 - éviter les champs `public` (sauf pour certaines constantes) afin notamment de ne pas contraindre fortement l'implémentation et limiter son évolution
-

Variables objet

- Une variable qui référence un objet permet d'accéder à ses membres visibles dans le contexte à l'aide de la notation :
`objet.membre`
- Affectation par défaut à la valeur `null` (sauf pour les variables locales)
- La création et l'initialisation d'objets passe typiquement par des méthodes particulières appelées **constructeurs**
 - ces méthodes portent le nom de leur classe
 - elles sont `public` si elles doivent être appelées depuis l'extérieur du package
 - elles peuvent prendre différentes séquences de paramètres (**surcharge** ↷)
 - elles renvoient (automatiquement) une nouvelle instance de la classe
 - elles sont invoquées à l'aide de l'opérateur `new`

```
1    Date uneDate = new Date(); // affectation de la
    référence du nouvel objet à une variable
2    // ...
3    System.out.println("Nous sommes le : ", new
    Date().toString()); // création d'objet anonyme
```

Variables objet sans objet

- Java autorise des variables objet à ne pas faire référence à un objet : elles contiennent alors la valeur `null`

🔗 que peut-il se passer à la compilation ? à l'exécution ?

```
1  Personne p = null;  
2  p.getNomComple() ;
```

🔗 que peut-on y faire ? >

```
1  Exception in thread "main"  
    java.lang.NullPointerException: Cannot invoke  
      "Personne.getNomComple()" because "< local1>"  
      is null  
2      at Test.main(Test.java:11)
```

📋 approches possibles >

- écrire des programmes sans bugs (👍)
- avoir une approche très défensive ? (ex. `if (p != null) ...` ; capture des exceptions `NullPointerException`) (💡)
- avoir un moyen d'exprimer des valeurs *optionnelles*
- avoir un système de types qui interdit les valeurs `null`

💡 qu'en penser ? >

- Java a proposé des évolutions en rapport :
 - amélioration de l'utilité des traces d'exception ([JEP 358: Helpful NullPointerExceptions](#))

- introduction d'un type pour des valeurs optionnelles : `Optional<T>` (🔗8) ↷
- introduction d'**annotations** (méta-données sur le code ↷) qui peuvent être utilisées par des frameworks/validateurs (ex. `@NotNull`, `@Nullable`)
- d'autres langages ont des types qui autorisent ou interdisent ces valeurs
 - ex. Kotlin : `Person` (n'autorise pas), `Person?` (autorise)

[✎ remontée à la source >](#)

Introduction des références Null dans ALGOL W (1965) "simply because it was so easy to implement [...] it may be perhaps a billion dollar mistake" (Tony Hoare)

Survenue de `NullPointerException`

Module `java.base`

Package `java.lang`

Class `NullPointerException`

```
java.lang.Object
  java.lang.Throwable
    java.lang.Exception
      java.lang.RuntimeException
        java.lang.NullPointerException
```

All Implemented Interfaces:

`Serializable`

```
public class NullPointerException
  extends RuntimeException
```

Thrown when an application attempts to use `null` in a case where an object is required. These include:

- Calling the instance method of a `null` object.
- Accessing or modifying the field of a `null` object.
- Taking the length of `null` as if it were an array.
- Accessing or modifying the slots of `null` as if it were an array.
- Throwing `null` as if it were a `Throwable` value.

Applications should throw instances of this class to indicate other illegal uses of the `null` object.

`NullPointerException` objects may be constructed by the virtual machine as if `suppression were disabled` and/or the stack trace was not writable.

Since:

1.0

[source](#)

Encapsulation des données

Schéma typique

- champs de donnée privés (`private`)
- méthodes publiques (`public`) d'accès en **consultation** (*getters*) et en **altération** (*setters*)

```
1  public class MaClasse {
2      private int monEntier;
3      // ...
4
5      // méthode de consultation
6      public int getMonEntier { return monEntier; }
7
8      // méthode d'altération
9      public boolean setMonEntier(int valeur) {
10         if ( condition_OK ) {
11             monEntier = valeur;
12             return true;
13         }
14         return false;
15     }
16 }
```

bénéfices

- les méthodes d'altération peuvent détecter (et traiter) des erreurs sur les changements d'états des objets
 - si la valeur d'un champ devient incorrecte, seule la méthode d'altération (et ses appels) doit être vérifiée (...)
-

② existe-t-il des situations où le principe d'encapsulation pourrait ne pas être respecté en dépit de l'utilisation de champs `private` et de *getters/setters*? >

Le **principe d'encapsulation** n'est pas respecté si une méthode d'accès retourne une référence vers un membre correspondant à un **objet altérable**

```
1 public class Personne {
2     private Date dateNaissance; // note: classe
    désormais obsolète
3     // ...
4     public Date getDateNaissance() {
5         return dateNaissance; // retourne la
    référence au champ privé
6     } // ...
7 }
```

```
1 Personne p = new Personne("Victor", "Hugo");
2 // ...
3 Date d = p.getDateNaissance();
4 System.err.println("Date: " + d.toString());
5 d.setTime(d.getTime() + (long) 1000 * 3600 * 24 *
    365);
6 // modification de la date de naissance de Victor
    Hugo (...)
7 d = p.getDateNaissance();
8 System.err.println("Date: " + d.toString());
```

📋 solutions possibles >

- avoir des objets **inaltérables** / **immuables** ↪
- retour de copies (**clones** ↪) pour les champs altérables

Privilèges d'accès

- Une méthode d'instance est invoquée sur un objet : cet objet est le **paramètre implicite** de l'appel de la méthode

```
d.setTime(demain); // 'd' est le paramètre implicite
```

- Une méthode peut accéder aux membres privés de l'objet qui l'a appelée

```
1 private String nom;  
2 // ...  
3 public String getNom() {  
4     return nom;  
5 }
```

- La notation **this** donne un accès au paramètre implicite depuis le code de la classe

❓ pourquoi, et dans quels cas utiliser **this** ? >

- quand il n'y a pas d'ambiguïté, les recommandations indiquent d'omettre ce que le contexte impose
- il y a ambiguïté si l'accès à un membre du paramètre implicite est en collision lexicale avec une variable, ex.

```
1 void ajoutePointsQuitus(final int pointsQuitus) {  
2     this.pointsQuitus += pointsQuitus;  
3 }
```

❓ une méthode peut-elle également accéder aux membres privés d'autres objets ? >

Oui : une méthode peut également accéder aux membres privés de n'importe quel objet de la classe de l'objet qui l'a

appelée

```
1  public class String {  
2      public int compareTo(String autreChaîne) {  
3          // la méthode a également accès aux champs  
          et méthodes privés  
4          // de 'autreChaîne' (même type)  
5          // ...  
6      } // ...  
7  }
```

Test d'égalité

L'opérateur `==` (resp. son inverse `!=`) teste si :

- deux expressions d'un type primitif ont des valeurs égales (resp. différentes)

```
1  int entier1 = 1, entier2 = 1;
2  // (entier1 == entier2) évalué à true
3  entier1 = 2 ;
4  // (entier1 == entier2) évalué à false car entier1 et
5  // entier2 n'ont plus la même valeur
```

- deux références d'objets sont égales (i.e. correspondent au même objet (resp. correspondent à des objets différents))

```
1  String chaine1 = new String("bonjour");
2  String chaine2 = new String("bonjour");
3  String chaine3 = chaine1;
4  System.err.println("chaine1 == chaine2 : " + (chaine1
    == chaine2 ? "vrai" : "faux")); // faux
5      System.err.println("chaine1 == chaine3 : " +
    (chaine1 == chaine3 ? "vrai" : "faux")); // vrai
```

Test d'égalité entre états d'objets

Pour tester plus généralement l'**égalité entre les états de deux objets**, il faut vérifier champ à champ que les données membres sont effectivement égales

❓ comment le faire de façon uniforme dans le langage ? >

On passe par la **redéfinition** d'une méthode définie au niveau d'un type parent de tous les autres types

- méthode `public boolean equals(Object autreObjet)` de la classe `java.lang.Object`
- ex. : `if (cetElève.equals(cetAutreElève))`

❓ et si on ne redéfinit pas equals ? >

Sans redéfinition, on utilise par défaut une version de `equals` reposant simplement sur la comparaison des références (`==`) (telle que définie dans `java.lang.Object`).

Redéfinition de `equals`

Propriétés de la méthode `equals`

Les spécifications du langage Java imposent les propriétés suivantes pour la méthode `equals`, pouvant être redéfinie au niveau de chaque classe

- **réflective** : `o.equals(o)` doit renvoyer vrai
 - **symétrique** : `o1.equals(o2)` doit renvoyer vrai ssi `o2.equals(o1)`
 - **transitive** : si `o1.equals(o2)` et `o2.equals(o3)` renvoient vrai, alors `o1.equals(o3)` renvoie vrai également
 - **cohérente** : des appels successifs doivent renvoyer la même valeur tant que les objets n'ont pas été modifiés
 - en outre, `o1.equals(null)` doit renvoyer faux
-

Approche pour la redéfinition de `equals`

1. Tester si les deux références ne sont pas égales (évite des calculs inutiles)

```
if (this == autreObjet) return true;
```

2. Si le paramètre explicite est `null`, retourner faux
3. Tester l'égalité des types des objets comme approprié

```
if (this.getClass() != autreObjet.getClass())  
return false;
```

4. Conversion du paramètre explicite en une variable du type de la classe

```
Classe objet = (Classe) autreObjet; ↩
```

5. Comparaison des champs un à un, avec `==` pour les types primitifs et `equals` pour les champs objets

```
return champ1 == objet.champ1 && champ2.  
(objet.champ2) ...
```

 Les IDEs permettent une définition automatique de cette redéfinition

 s'assurer qu'elle est bien mise à jour si la définition d'une classe évolue

② que se passe-t-il si l'on pense faire une redéfinition (ex. de `equals`) mais que la signature diffère de celle de la méthode qu'on souhaite redéfinir ? >

- Par exemple, la redéfinition dans `java.lang.Object` de :
 - `public boolean equals (Object o2)`
dans `Etudiant` par :
 - `public boolean equals (Etudiant e2)`
crée simplement une **surcharge** de la méthode `equals` et mènera à un appel inattendu à l'exécution
- Pour s'en protéger, il faut indiquer explicitement au compilateur l'intention de redéfinition
 - cela se fait au moyen d'une **annotation** (☞ 5.0)
: `@Override`
 - ex. `@Override public boolean equals (Object o2)`
 - le compilateur renvoie une erreur si la redéfinition ne peut être vérifiée

```
1 Test.java:12: error: method does not override or
  implement a method from a supertype
2     @Override public boolean equals (Test t) {
```

≡ Exemple de redéfinition de `equals()`

```
1  public class Personne {
2      // ...
3      @Override public boolean equals(Object objet)
4      {
5          if (this == objet) return true;
6          if (objet == null) return false;
7          if (objet.getClass() != this.getClass())
8              return false;
9          Personne autrePersonne = (Personne) objet;
10         if
11             (this.prénom.equals(autrePersonne.prénom)
12              && this.nom.equals(autrePersonne.nom))
13             return true;
14         return false;
15     }
16     // ...
17 }
```

Copie de références d'objets

🔗 Copier une référence d'objet ne crée pas un nouvel objet, juste une nouvelle référence vers le même objet

```
1  Date date1 = new Date(); // rappel: classe désormais
   obsolete
2  Date date2 = date1;
3
4  // affiche la première valeur de l'objet date1
5  System.err.println(date1);
6
7  // modification de l'objet date1
8  date1.setTime((long)date1.getTime() + 1000 * 3600 *
   24);
9
10 // affiche la nouvelle valeur de l'objet date1
11 System.err.println(date1);
12
13 // affiche la valeur de l'objet référencé par date1
14 // lors de l'affectation date2 = date1 (i.e. celui
15 // actuellement référencé par date2)
16 // (note : l'ancienne valeur est perdue)
17 System.err.println(date2);
```

Copie d'états d'objets

- Pour obtenir une copie indépendante d'un objet, on passe par l'opération de **clonage** ↪
 - définie par une méthode `protected` ↪ de la classe

`java.lang.Object`

```
1  Date date1 = new Date();
2  Date date2 = (Date)date1.clone(); // redéfinie au
   niveau de la classe Date
3  System.err.println(date1);
4  date1.setTime((long)date1.getTime() + 1000 * 3600 *
   24);
5
6  // affiche la nouvelle valeur de l'objet date1
7  System.err.println(date1);
8
9  // affiche la valeur de l'objet référencé par date1
   lors
10 // de l'affectation date2 = date1 (car date2 n'a pas
   été
11 // modifiée), i.e. l'ancienne valeur de date1
12 System.err.println(date2);
```

✍ Il existe des alternative, par exemple les constructeurs de copie (qui prennent en paramètre une instance de leur classe)

Passage de paramètres aux méthodes

🔗 Les paramètres sont passés *par valeur*

- une méthode ne peut donc jamais modifier la valeur d'un paramètre effectif (elle n'en reçoit qu'une copie)
- toutefois, lorsqu'un paramètre correspond à un objet, il est possible de "modifier" cet objet *via* la copie de sa référence (si l'on dispose de l'interface appropriée)

```
1 public void échangeNomEtPrénom(Personne p) {  
2     String nouveauPrénom = p.getNom();  
3     p.setNom(p.getPrénom());  
4     p.setPrénom(nouveauPrénom);  
5 }
```

❓ pourquoi ne peut-on pas parler d'un passage de paramètres *par référence* ?



- Par exemple, l'échange des valeurs de deux objets passés en paramètres ne fonctionne pas au niveau des paramètres effectifs

```
1 public void échange(Etudiant étu1, Etudiant étu2)  
2 {  
3     Etudiant temp = étu1; étu1 = étu2; étu2 =  
4     temp;  
5     // les paramètres effectifs ne sont pas  
6     modifiés, seules les variables locales le sont  
7 }
```

Surcharge de méthodes

- Des méthodes d'une même classe peuvent partager le même nom mais avec des paramètres différents : on parle de **surcharge de méthode** 📦

⚠️ le compilateur génère une erreur s'il ne peut pas identifier la méthode devant être effectivement appelée (résolution de surcharge)

📋 permet de définir un même traitement pour des paramètres différents

ex. la classe `java.lang.String` contient différentes méthodes `indexOf` (position d'un caractère dans une chaîne)

- `int indexOf(int char)`
- `int indexOf(int char, intfromIndex)`, etc.

≡ notamment utilisée pour définir plusieurs constructeurs pour une classe

```
1  public Etudiant(String lePrénom, String leNom) {
2      // appel du constructeur surchargé
3      this(lePrénom, leNom,
4          Etudiant.ID_NON_ATTRIBUEE);
5  }
6  public Etudiant(String lePrénom, String leNom, int
7      id) {
8      // appel du constructeur de la classe parente
9      super(lePrénom, leNom);
10     identifiant = id;
11 }
```

② serait-il possible d'avoir deux méthodes ne différant que pas leur type de retour >

- dans une même classe, non !
- ? et lors de redéfinition dans des sous-types ? ↩

Fonctions variadiques (*varargs*)

- Java permet d'appeler des méthodes avec un nombre variable de paramètres (☞ 5.0) (*varargs*)
 - ex. méthode `printf` de `PrintStream`
 - `public PrintStream printf(String format, Object... args)`
 - le deuxième paramètre est en fait un tableau de `Object`
 - concerne (au plus) le dernier paramètre de la méthode
 - permet de mettre des listes de paramètres de taille non connue à la compilation de la méthode

```
1 public static int max(int... valeurs) {  
2     int max = Integer.MIN_VALUE ;  
3     for (int v : valeurs)  
4         if (v > max) max = v ;  
5     return max ;  
6 }  
7 // ...  
8 int valeurMax = max(5, 17, 3, 12, 2, 38);
```

Méthodes privées (`private`)

Les méthodes peuvent être déclarées `private` : elles ne peuvent être invoquées que sur des objets de leur classe

```
private void cuisineInterne ()
```

Cas d'usage

- méthodes qui reflètent trop directement le fonctionnement interne d'une classe
- découpage en unités de résolution (méthodes) de taille appropriée n'ayant pas vocation à être appelée directement depuis l'extérieur de la classe

Points importants

- si l'implémentation de la classe change (par héritage), une méthode privée peut ne plus être nécessaire ou adaptée (cf. déclarations `protected` ↔)
 - en encapsulant les détails d'implémentation, la classe peut évoluer sans que son interface publique ne soit modifiée
-

Champs statiques (static)

📄 Les champs peuvent être déclarés static

- il n'en existe qu'une copie par classe, qui n'est donc pas attachée à une instance particulière
- le champ ne nécessite (en fait) pas d'instance pour exister
- permet donc d'avoir un champ unique commun à l'ensemble des instances, ex.

```
1 private static int idSuivant = 1;
2 // ...
3 public void setID() {
4     id = idSuivant++;
5 }
```

Méthodes statiques (`static`)

- Les méthodes peuvent être déclarées `static` ("de classe")
 - on les invoque avec le nom de leur classe : `Math.pow(x, a);`
 - elles ne peuvent accéder directement aux champs et aux méthodes d'instances de la classe (*ce qui nécessite... une instance*)

Cas d'usage

- une méthode n'a pas besoin d'accéder à l'état d'un objet (tous ses paramètres nécessaires sont transmis en paramètres explicites)
 - une méthode n'a besoin d'accéder qu'à des membres statiques de la classe
 - ex. : méthodes dites **factory** ↪ qui retournent des objets d'un type (alternatives aux constructeurs)
-

Construction d'objets

Rôle : définir l'état initial des objets instanciés

Procédure de construction d'un objet

1. initialisation des champs à leur valeur par défaut
 2. (éventuellement) exécution des initialisateurs de champs et blocs d'initialisation (dans leur ordre d'apparition)
 3. (éventuellement) appel d'un autre constructeur
 4. exécution du constructeur
-

- Possibilité de **surcharge** des constructeurs : plusieurs versions, avec des paramètres différents

```
1 public Etudiant() { ... }  
2 public Etudiant(String nom) { ...}  
3 public Etudiant(String nom, int id) { ...}
```

- Possibilité d'appel d'un autre constructeur de la même classe (première instruction du constructeur) : `this()`

```
1 public Etudiant(String nom) {  
2     this(nom, Etudiant.UNKNOWN_ID);  
3 }
```

Construction d'objets: initialisation des champs

Initialisation implicite : `0`, `false`, ou `null` (références)

⚠ les variables locales doivent toujours être initialisées explicitement

Initialisation explicite

- valeur littérale au niveau de la déclaration du champ

```
1 private String nom = "A. Nonyme";
```

- valeur d'appel de méthode `static` au niveau de la déclaration

```
1 class Etudiant {  
2     private int id = affecteID();  
3     static int affecteID() { ... }
```

- bloc d'initialisation (éventuellement `static`)

```
1 class Etudiant {  
2     static {  
3         Random generateur = new Random();  
4         idSuivante = generateur.nextInt(100);  
5     }
```

- (et bien sûr) affectations dans le constructeur

```
1 public Etudiant(String nom, int id) {  
2     this.nom = nom; this.id = id;  
3 }
```

Construction d'objets : exemple

```
1  public class Employé {
2      private static int idSuivante;
3      private int id;
4      private String nom;
5      private double salaire;
6
7      // bloc d'initialisation statique : exécuté une
      seule fois
8      static {
9          Random generateur = new Random();
10         idSuivante = generateur.nextInt(100);
11     }
12
13     // bloc d'initialisation d'objet : pour chaque
      nouvel objet
14     {
15         id = idSuivante++;
16     }
17
18     public Employé(String nom, double salaire) {
19         this.nom = nom; this.salaire = salaire;
20     }
21
22     public Employé(double salaire) {
23         this("employé n°" + id, salaire);
24     }
25 }
```

Fin de vie des objets

⚠ L'absence de libération d'objets devenus non nécessaires peut mener à une fuite mémoire

- La personne qui développe en Java est déchargée en grande partie de la gestion des ressources mémoires
 - elle n'a donc pas à désallouer explicitement la mémoire réservée pour les objets ↔
- Un processus de **ramasse-miettes** (*garbage collecting*) 🗑 est responsable de récupérer l'espace mémoire associé à des objets n'étant plus référencés
- Le fonctionnement du ramasse-miettes est régi par un algorithme, mais il est possible de demander (sans garantie) son exécution

```
1  Runtime runtime = Runtime.getRuntime();  
2  runtime.gc();
```

✍ D'autres langages orientés objet (ex. C++) requièrent la définition de méthodes dites destructeurs

Héritage et polymorphisme

Réutilisation de code

🔥 Les bonnes pratiques de développement suggèrent de réutiliser autant que possible le code déjà développé

i.e. écrit, testé, éprouvé, optimisé, etc.

📋 Lorsqu'une classe existante couvre (partiellement !) les besoins d'une nouvelle classe, plusieurs scénarios sont possibles

1. la nouvelle classe possède un champ propre correspondant à une instance de l'autre classe : c'est une **composition** ➡
 - ex. : chaque instance de `Etudiant` a sa propre `adresse` (classe `Adresse`)
2. la nouvelle classe possède un champ qui référence une instance déjà existante de l'autre classe : c'est une **agrégation** ➡
 - ex. chaque instance de `Etudiant` a une `institution` (classe `Institution`) partagée
3. la nouvelle classe est une spécialisation de l'autre classe, elle reprend son interface et son implémentation (avec des restrictions de visibilité) : c'est de l'**héritage** ➡

ex. un `Etudiant` est une `Personne`, on peut vouloir lui appliquer tout ce qui est applicable à ce type (hors méthodes privées)

La composition

ne pas être partageur

- Dans la **composition**, une instance propre d'une autre classe est utilisée pour définir la nouvelle classe *via* un champ
 - le principe d'encapsulation réserve cet objet aux objets de la classe : ils en possèdent leur propre copie (qui disparaîtra avec eux)
 - cet objet doit être instancié par la classe (par exemple, dans l'un de ses constructeurs)

```
1  public class Personne {
2      private Adresse adresse;
3      public Personne(String prénom, ..., String rue,
4          String ville, String codePostal) {
5          // ...
6          adresse = new Adresse(rue, ville, codePostal)
7      };
8      // méthode publique d'altération
9      public void setAdresse(String rue, String ville,
10         String codePostal) {
11         adresse = new Adresse(rue, ville,
12             codePostal); // ...
13     }
14 }
```

L'agrégation

être partageur

- Dans l'**agrégation**, une instance d'un objet existant (externe à la classe) est utilisée pour définir une nouvelle classe *via* un champ
 - cet objet peut donc évoluer indépendamment des objets de la nouvelle classe
 - les objets de la nouvelle classe ont toutefois bien accès à (au moins) l'interface publique de cet objet

```
1  public class Etudiant ... {
2      // ce champ correspond à un objet externe
3      private Institution institution;
4
5      public Etudiant(..., Institution institution) {
6          // ...
7          this.institution = institution;
8      } // ...
```

L'héritage (`extends`)

- Dans d'autres contextes, on souhaite définir une nouvelle classe :
 - qui reprend les fonctionnalités d'une classe existante et est en lien logique direct avec elle
 - mais qui peut définir de nouveaux comportements, et/ou redéfinir des comportements existants
 - cette *spécialisation* peut se faire grâce à la notion d'héritage
 - définition des "différences" entre deux classes (ajouts, redéfinitions)
 - ce qui est plus "général" se trouve dans la classe dont on hérite, et ce qui est plus "spécifique" dans la classe qui hérite

📋 Terminologie de la relation entre classes : si la classe `B` hérite de la classe `A`, on dit (notamment) que :

- `A` est une **super-classe/classe parente/classe mère** de `B`
- `B` est une **sous-classe/classe dérivée/classe fille** de `A`

- Utilisation du mot-clé `extends`

```
1 class B extends A { // la classe B hérite de la
    classe A
2     // champs ajoutés et méthodes ajoutées ou
    redéfinies
3 }
```

Accès protégé (`protected`)

- Lors de la définition d'une classe par héritage, certains champs et certaines méthodes sont accessibles depuis les classes filles :
 - ceux qui sont `public` (attendu) et `protected` (prévu ainsi) (ainsi que *package-protected* le cas échéant)

⚠ pas les champs qui sont `private` : ils n'appartiennent vraiment qu'à la super-classe

📋 Accès protégé (`protected`)

- le mot-clé `protected` indique donc une visibilité à toutes les sous-classes (et donc également à tout le package)
 - cela permet d'accéder à des méthodes sensibles lorsqu'on a une relation d'héritage
 - ex : `protected Object clone()` de `java.lang.Object`
 - cela permet également à des sous-classes d'accéder directement à des champs `protected` définis dans une super-classe
-

⚠ Encapsulation des données et héritage

- Pour ne pas violer le principe d'encapsulation des données, une sous-classe ne pourra accéder qu'aux champs protégés appartenant à des objets de sa propre classe, pas à des objets d'une super-classe
- Cela permettrait sinon de détourner l'usage de l'héritage pour avoir accès aux données protégées d'autres classes

```
1  class A {
2      protected Type champ; // champ protégé de la
    superclasse
3  }
4  class B extends A {
5      // ...
6      champ = valeur; // ne peut faire référence
    qu'au champ de la classe B
7      A aMaisPasB = new A();
8      aMaisPasB.champ = valeur; // accès interdit à
    celui d'une instance de A (qui ne serait pas un B)
9      // ...
10 }
```

Accès aux méthodes des super classes

- Une méthode définie dans une super classe peut être invoquée directement depuis une classe fille (... si elle est *accessible*)
- Si une méthode d'une super classe est redéfinie dans une classe fille (`@Override`), il reste possible de l'invoquer depuis la classe fille (... si elle est *accessible*) à l'aide du mot-clé `super`

```
1  public class Personne {
2      private String prénom, nom;
3
4      public String getNomComplet() {
5          return prénom + " " + nom;
6      } // ...
7  }
8
9  public class Etudiant extends Personne {
10     private boolean estInscrit;
11
12     @Override public String getNomComplet() {
13         // accès impossible aux champs prénom et nom
14         // accès direct impossible à
15         // car la méthode est masquée par la méthode
16         // en cours de redéfinition
17         return super.getNomComplet() + "(inscription
18             : " + (estInscrit ? "oui" : "non") + ")";
19     } // ...
20 }
```

Construction d'objets et héritage (`super`)

- Possibilité d'appeler un constructeur de la classe parente avec certains paramètres
 - utilisation du mot-clé `super`
 - permet à une sous-classe qui n'a pas accès aux champs privés de ses classes parentes de les initialiser *via* un constructeur de la classe parente

⚠ L'invocation du constructeur de la classe parent doit être la première instruction du constructeur

```
1 public class Manager extends Employé {
2     public Manager(String nom, double salaire) {
3         super(nom, salaire);
4         // ...
5     }
6 }
```

❓ que se passe-t-il si un constructeur n'appelle pas de constructeur de sa classe parente ?



Si un constructeur d'une sous-classe n'appelle pas explicitement un constructeur de la classe parente, le constructeur par défaut (i.e. sans paramètres) est appelé

⚠ Si la classe parente ne possède pas de constructeur par défaut, le compilateur indique une erreur

```
1  class A {  
2      private int a;  
3      public A(int a) { this.a = a; }  
4  }  
5  
6  public class B extends A {  
7      public B() { }  
8  }
```

```
1  A.java:3: error: no suitable constructor found  
   for A(no arguments)  
2      public B() { }  
3          ^  
4      constructor A.A(int) is not applicable  
5          (actual and formal argument lists differ in  
           length)
```


Héritage et redéfinition (`@Override`)

- Une méthode d'une super-classe qui est visible depuis une sous-classe peut (évidemment) être réutilisée telle quelle
- Possibilités de **redéfinition de méthode** `>>` (`@Override`)
 - on peut faire référence à la méthode de la super-classe (`super`) et ajouter ce qui est spécifique
 - on définit un code nouveau adapté à la classe spécialisée

Points importants

? **quelle méthode est invoquée :** >

```
etudiant.getNomComplet()
```

Question de **résolution de méthode** `>>`

? **est-il possible de "supprimer" des champs ou des méthodes**

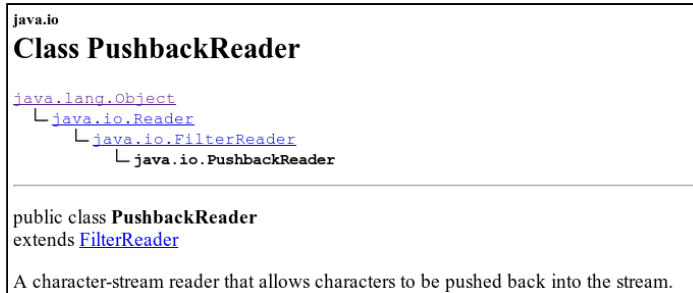
? **est-il possible de modifier la visibilité ou le type de retour lors d'une redéfinition de méthode ?**

? **devrait-il être possible d'empêcher la redéfinition d'une méthode, et pourquoi ?** >

- On peut empêcher la redéfinition de méthodes lors de l'héritage en spécifiant les méthodes non redéfinissables par le mot-clé `final`.
- Cela garantit une implémentation particulière dans les sous-classes.

Chaînes d'héritage

- Il est possible d'hériter de classes dérivées, sans limite (théorique) sur la profondeur de la hiérarchie de classes ainsi définie
 - la **chaîne d'héritage** est le chemin d'accès d'une classe particulière vers ses ancêtres



② devrait-il être possible d'empêcher l'héritage depuis une classe, et pourquoi ? >

Il est possible d'**interdire l'héritage d'une classe** (comme la redéfinition d'une méthode) :

- on utilise le modificateur `final`
`final class MaClasseNonDérivable`

② devrait-il être possible de limiter les sous-classes d'une classe, et pourquoi ? >

Il est possible de **limiter les classes pouvant hériter d'une classe** pour des **classes scellées** (*sealed classes*) 🔒 (§ 15) ↩

- on utilise le mot-clé `permits`
`sealed class Classe permits SousClasse1 [, ...]`

La super classe `java.lang.Object`

java.lang

Class Object

java.lang.Object

```
public class Object
```

Class Object is the root of the class hierarchy. Every class has Object as a superclass. All objects, including arrays, implement the methods of this class.

Since:

JDK1.0

- Au sommet de la chaîne d'héritage de toute classe, la classe `java.lang.Object` définit plusieurs méthodes publiques qui sont donc héritées par toute nouvelle classe, notamment :
 - `public boolean equals(Object autreObjet)` : indique (par défaut) si deux objets correspondent physiquement (en mémoire) au même objet
 - `public String toString()` : renvoie une description sous forme de chaîne de caractères de l'état d'un objet
 - `public Class getClass()` : renvoie la classe d'un objet
 - `protected Object clone()` : crée une copie physique d'un objet

🤔 pourquoi `clone` a-t-il la visibilité `protected` ?

Héritage multiple en Java ?

② devrait-il être possible pour une classe de pouvoir hériter > de plusieurs classes ?

Il n'est pas possible pour une classe d'hériter *simultanément* de plusieurs classes en Java

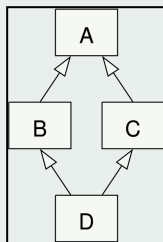
- Java préfère la notion d'**interface** ⇨ (implémentations multiples autorisées) ⇨

✍ l'héritage multiple est permis par d'autres langages orientés objet (ex. C++)

📅 situation problématique: *problème du diamant* >

Dans une situation où :

- des classes B et C héritent d'une classe A
- une classe D hériterait *simultanément* de B et C
- une méthode m est définie dans A
- comment résoudre un appel à m sur un objet de D ?



Configurations

- si m est uniquement redéfinie dans B ou dans C : ☑
- si m est redéfinie au niveau de D (sans invocation de `super.m`) : ☑

- si m est redéfinie dans B et dans C : résolution de méthode impossible (sans syntaxe spécifique) : $\otimes$
-

Polymorphisme

🔥 Principe de substitution

- on peut utiliser un objet d'une sous-classe (ex. `Etudiant`) chaque fois que le programme attend un objet d'une super-classe (ex. `Personne`)
- les variables objets sont donc **polymorphes** ➡

```
1  Personne p;  
2  p = new Personne(...); // OK comme attendu  
3  p = new Etudiant(...); // possible
```

⚠ On ne peut bien sûr pas utiliser ce qui est propre aux sous-classes lorsqu'on manipule une variable de la super-classe

```
1  Personne p = ...;  
2  p.inscrire(); // pas défini (erreur de compilation)
```

Transtypage de références

- Possibilité de transtypage de référence d'objet *lorsque pertinent*

```
1  if(p instanceof Etudiant) {  
2      ((Etudiant)p).inscrire();  
3      // ...  
4  }
```

- Variante `instanceof` avec **pattern matching** (§14)

```
1  if (p instanceof Etudiant etudiant) {  
2      etudiant.inscrire();  
3      // ...  
4  }
```

Erreur de transtypage à l'exécution

(`ClassCastException`)

java.lang

Class `ClassCastException`

[java.lang.Object](#)

└ [java.lang.Throwable](#)

└ [java.lang.Exception](#)

└ [java.lang.RuntimeException](#)

└ `java.lang.ClassCastException`

All Implemented Interfaces:

[Serializable](#)

public class `ClassCastException`

extends [RuntimeException](#)

Thrown to indicate that the code has attempted to cast an object to a subclass of which it is not an instance. For example, the following code generates a `ClassCastException`:

```
Object x = new Integer(0);
System.out.println((String)x);
```

Types de retour covariants

② devrait-il être possible de modifier le type de retour d'une > méthode redéfinie ?

- La **redéfinition de méthode** (`@Override`) consiste à redéfinir une méthode dans une sous-classe avec la même signature (même nom et mêmes paramètres)
- Une sous-classe peut uniquement modifier le type de retour **par un sous-type du type d'origine** (🔗5)
 - on parle de **types de retour covariants** 🗨️

≡ exemple : `class Etudiant extends Personne`

```
1 public class Personne {  
2     // ...  
3     public Personne getAssociéPrincipal()  
4     { // ...  
5     }  
6 public class Etudiant extends Personne {  
7     // ...  
8 }
```

② `@Override public Personne` >
`getAssociéPrincipal()`
possible ✔️

② `@Override public Etudiant Personne` >
`getAssociéPrincipal()`
également possible ✔️

② @Override public Animal Personne >
getAssociéPrincipal ()

impossible ⊗

Résolution des appels de méthodes

② Détermination de la méthode à appeler pour un appel effectif :

`o.m(param)` , avec `o` instance de la classe `A` , référencée par une variable qui peut être d'un super-type de `A`

📋 Étapes de la liaison dynamique pour l'invocation

1. Énumération des méthodes `m` de la classe `A` et des méthodes publiques `m` des super-classes de `A`
2. **Résolution de surcharge** : si plusieurs listes de paramètres parmi les méthodes précédentes sont compatibles avec `param` (notamment avec les conversions de types possibles), le compilateur renvoie une erreur
3. **Liaison statique** : si la méthode est un constructeur, ou `private` (privée), `static` (de classe) ou `final` (non redéfinissable), le compilateur sait alors quelle méthode effectivement invoquer
4. **Liaison dynamique** : la méthode à appeler dépend sinon du type effectif (réel) du paramètre implicite `o` , cette résolution est faite à l'exécution
 - si `A` ne définit pas la méthode `m` recherchée, celle-ci est recherchée dans sa classe mère, puis dans sa super-classe grand-mère, etc.

☞ Polymorphisme

"Once you know that all method binding in Java happens polymorphically via late binding, **you can always write your code to talk to the base class, and know that all the derived-class cases will work correctly using the same code.**

Or put it another way, **you send a message to an object and let the object figure out the right thing to do.**" -- Bruce Eckel,
Thinking in Java

Hiérarchie de classes

Plus on remonte dans la hiérarchie des classes, plus les définitions représentent des entités "abstraites"

🔗 y a-t-il un intérêt à définir des classes qu'on ne pourrait pas instancier ? >

- Une classe ancêtre peut jouer le rôle d'un "moule" pour des classes dérivées qui n'a pas vocation à avoir des instances de son type
 - On aurait tout de même intérêt à factoriser les champs et les définitions de méthodes qui seraient communs à toutes les (futures) classes dérivées
 - Mais certaines méthodes ne pourront pas être *définies* à un certain niveau d'abstraction
-

Classes abstraites

🔗 On peut *déclarer* des méthodes dans une classe où elles ne pourraient être *définies*

- elles seront dites **abstraites** 🔗 (`abstract`)
- ces méthodes participent alors au contrat pour tous les objets de ce type et de ses (futurs) sous-types (distinctions de visibilité ↪)

🔗 qu'en est-il de `java.lang.Object` ? >

Cette classe est déclarée abstraite, mais n'a pas de méthodes abstraites (on dispose d'une implémentation pour chacune d'elles).

🔗 qu'est-ce qui caractérise une (sous-)classe *concrète* ? >

- Une classe **concrète** 🔗 définit toutes ses méthodes (et n'est pas qualifiée d'abstraite) : on peut en instancier des objets (si proposé)
- Une (sous-)classe concrète devra fournir une définition pour les méthodes abstraites de ses ancêtres *qui seraient encore abstraites* (`@Override`)

🔗 quelles peuvent être les visibilités d'une méthode abstraite ?

≡ Exemple: les inscriptions des étudiants pourraient être particulières à leur institution

- il devrait bien être possible d'invoquer sur les objets d'un sous-type de `Etudiant` une méthode `inscrire()`
- mais celle-ci ne pourrait pas être définie au niveau de la classe `Etudiant` : on y déclare donc la méthode comme **abstraite**
- la classe `Etudiant`, qui ne définit donc pas toutes ses méthodes, est elle-même **abstraite** (mot-clé `abstract` sur la classe)

```
1  public abstract class Etudiant extends Personne {
2      protected boolean estInscrit = false;
3      // la méthode ne sera pas définie ici, mais on
      s'assure
4      // que tout objet d'un sous-type concret de
      Etudiant
5      // disposera d'une implémentation de cette
      méthode
6      public abstract boolean inscrire();
7
8      // ...
9  }
```

② peut-on créer des instances de classes abstraites ? >

La machine virtuelle ne saurait pas trouver à l'exécution certaines méthodes à exécuter.

② peut-on/doit-on tout de même y définir des constructeurs ? >

Oui, car ces classes ont vocation à être héritées, et donc il faut pouvoir invoquer leurs constructeurs pour ce qui concerne leur type.

- Les **classes abstraites** peuvent (bien sûr) posséder des données (concrètes) et définir des méthodes concrètes
- Les méthodes abstraites jouent donc le rôle d'*emplacements* et devront être implémentées tôt ou tard dans les sous-classes (assurément pour les classes concrètes)

```
1  public class EtudiantPolytech extends Etudiant {
2      // ...
3      @Override
4      public boolean inscrire() { ...
5          // définition concrète : la classe peut ne
           plus être abstraite
6          // ...
7      }
8  }
```

① Une sous-classe qui définit toutes les méthodes abstraites de sa chaîne d'héritage (et n'en introduit pas) *peut* ne plus être abstraite

② peut-on avoir des variables objets de types abstraits ?

- Rappel : on peut avoir des variables objets qui reçoivent des objets de leurs sous-types

```
Etudiant étudiant = new EtudiantPops("Jules",  
"Verne");
```

② ... >

- Si l'on dispose d'une instance (d'un type concret, donc), on peut appeler une méthode déclarée `abstract` sur une variable d'un type abstrait : on est certain qu'une définition sera accessible

```
1 Etudiant[] étudiants = { new  
  EtudiantPOPS("Jules", "Verne"), new  
  EtudiantIUT("Victor", "Hugo") } ;  
2 for (Etudiant étudiant : étudiants)  
3     étudiant.inscrire(); // la méthode est  
  résolue à l'exécution
```

② pourrait-on et pour quels intérêts généraliser le principe des méthodes abstraites à un type ?

Les interfaces (`interface`)

🔗 Rôle des interfaces 🔗

Préciser ce que les classes qui les *implémenteront* devront pouvoir faire (les messages que leurs objets pourront recevoir), sans préciser comment (...↪) : c'est un contrat

- (ce que font les classes abstraites pour leurs méthodes abstraites)

📋 Les interfaces définissent des types

- ce ne sont toutefois pas des **classes** : introduction du mot-clé `interface`
- notion d'**implémentation** (`implements`), qui définit bien toutefois des super-types (comme pour l'héritage)
- possibilité de tester le fait qu'un objet implémente une interface particulière (i.e. qu'il est de son type) à l'aide de l'opérateur `instanceof` :

```
if (étudiant instanceof Comparable) { // ...
```
- les interfaces n'ont pas d'instances (on invoquera leur méthode sur des objets de classes qui ne sont pas connues) : on ne peut donc pas définir le corps de leurs méthodes (...↪) ni y déclarer de champs d'instance

② quels pourraient être les niveaux de visibilité pour les méthodes des interfaces ? >

- `public` assurément
- ② `protected`

② peut-on toutefois avoir certaines définitions de méthodes dans les interfaces ? >

Oui (depuis [JDK 8](#))

- méthodes `default` ([JDK 8](#))
 - permettent de modifier une interface sans impacter le code existant qui les utilisent déjà (compatibilité avec d'anciennes versions de l'interface qui n'imposaient pas ces méthodes)
 - lors d'une implémentation, possibilité de ne pas mentionner la méthode `default`, de la redéclarer (la rend `abstract` à nouveau), de la définir
 - (en cas d'implémentation de plusieurs interfaces définissant la même méthode par défaut, nécessité de fournir une implémentation dans la classe (sinon erreur de compilation))
- méthodes `private` ([JDK 9](#))
 - permettent de réutiliser/factoriser du code dans les méthodes `default`
- méthodes `static` ([JDK 8](#))
 - similaires aux méthodes `default` mais ne pouvant pas être redéfinies dans les classes implémentantes (elles appartiennent au type de cette interface)

- garantissent donc une implémentation particulière d'une méthode pour cette interface

② peut-on toutefois avoir certains champs dans les interfaces ? >

- oui, des champs `public static final`
ex. `public static final int VAL_MAX = 20;`
- la spécification du langage déconseille toutefois de préciser les mots-clés redondants (i.e. imposés par le contexte), on utilisera donc :

```
int VAL_MAX = 20; // pas un champ d'instance dans  
le contexte d'une interface
```

② peut-on avoir des variables de type interface ? >

```
1 public class Etudiant implements Comparable { /*  
   ... */ }  
2 Comparable c = new Etudiant("Honoré", "de  
  Balzac"); // OK
```

② peut-on implémenter simultanément plusieurs interfaces dans une même classe ? >

Oui, et donc les objets seront de tous les types concernés (mais... ↩).

📄 Implémentation d'interfaces (`implements`)

- La définition d'une classe peut *implémenter* une interface

```
class Classe implements Interface { // ...
```

- Il est possible d'implémenter simultanément plusieurs interfaces

```
class Classe implements Interface1, Interface2 { //  
...  
}
```

- Par nature, les méthodes déclarées par une interface sont

`public`

- (possibilité d'avoir des méthodes `private` dans les interfaces, en lien avec l'introduction de méthodes par défaut (`default`) (🔗9))

🔗 Access control is all about hiding implementation details. An interface has no implementation to hide.

Déclaration d'interface (`interface`)

- Déclaration avec le mot-clé `interface`
- Une interface (`public`) par fichier `.java` qui porte son nom
- Une interface peut être définie par héritage d'une ou plusieurs autres interfaces (mot-clé `extends`)

```
public interface I1 extends I2, I3 { // ...
```

≡ Exemple : tri d'objets

La méthode `static void sort(Object[] a)` de `java.util.Arrays` trie un tableau selon un ordre défini localement entre deux objets

- chaque objet du tableau doit implémenter une méthode particulière, `public int compareTo(Object autreObjet)`, déclarée dans l'interface `java.lang.Comparable`
 - `ClassCastException` si le tableau contient des éléments non comparables entre eux, i.e. non manipulables comme instances du type `Comparable`
- définir une classe comme implémentant l'interface `Comparable` garantit que celle-ci dispose(ra) effectivement d'une définition pour cette méthode `compareTo`

```
1  public class Personne implements Comparable {
2
3      // note : il existe maintenant une version
      générique
4      // Comparable< T > (vu plus loin)
5      public int compareTo(Object autreObjet) {
6          Personne autrePersonne =
            (Personne) autreObjet;
7          int comparaisonNoms =
            this.nom.compareTo(autrePersonne.nom);
8          if (comparaisonNoms != 0) return
            comparaisonNoms;
9          return
            this.prénom.compareTo(autrePersonne.prénom);
10     }
11
12     // ...
```

```
13
14     Personne[] gens = new Personne[4];
15     gens[0] = new Personne("Marie", "Curie");
16     gens[1] = new Personne("Françoise", "Barré-
Sinoussi");
17     gens[2] = new Personne("Anne", "L'Huillier");
18
19     System.out.println(java.util.Arrays.toString(gens))
20     java.util.Arrays.sort(gens);
21
22     System.out.println(java.util.Arrays.toString(gens))
```

≡ Exemple : clonage d'objets

- Le **clonage** crée une copie d'un objet, initialement égale à l'objet, mais dont l'état peut diverger avec le temps
 - on ne veut donc pas simplement copier une référence
 - on ne veut pas non plus (en général) que la copie d'un objet soit obtenue par copie des références des champs objets (*shallow copy*)
- Utilisation de l'interface `Cloneable`
 - redéfinition de la méthode `protected Object clone()` de `Object` (mais avec la visibilité `public` ↷)
 - l'interface `Cloneable` ne fait que servir de "marqueur" (elle ne définit rien directement)
 - une erreur (exception `CloneNotSupportedException`) sera levée si un clonage (avec cette méthode) est demandé sans implémentation de cette interface
- On peut faire appel à la méthode de clonage de la super classe
 - par défaut, dans la classe `Object`, recopie des champs de types primitifs et des références d'objets (convient pour les champs non altérables)
 - pour les champs correspondant à des objets altérables, il faut demander leur clonage

```
1 public class Personne implements Cloneable {
2
3     public Personne clone() throws
      CloneNotSupportedException {
4         Personne clone = (Personne) super.clone();
5         clone.dateNaissance =
      (Date) dateNaissance.clone();
6         return clone;
      }
```

```
7      }  
8      // ...  
9  }
```

Classes anonymes

- On a parfois besoin d'une classe aux propriétés particulières (... *qui implémente une interface*) très localement
- Possibilité d'avoir recours à une **classe locale** 🗨
 - (différent de la notion de **classe interne** 🗨 ↩)

```
1 public static SéquenceEntiers entiersAléatoires(int
   min, int max) {
2     class SéquenceAléatoire implements
   SéquenceEntiers {
3         // méthodes requises par l'interface
   SéquenceEntiers
4         public int next() { return ... }
5         public boolean hasNext() { return true; }
6     }
7     return new SéquenceAléatoire();
8 }
```

- Possibilité de définir directement une classe implémentant l'interface sans la nommer : une **classe anonyme** 🗨
 - (introduction des **expressions lambda** (🔗 8) ↩)

```
1 public static SéquenceEntiers entiersAléatoires(int
   min, int max) {
2     return new SéquenceEntiers() { // pas un 'new'
   sur une interface!
3         public int next() { return ... }
4         public boolean hasNext() { return true; }
5     }
6 }
```

- De même, il est possible de définir localement des **(sous-)classes anonymes** qui héritent d'une classe particulière
- Cela permet, par exemple, d'enrichir un comportement particulier pour ce type

```
1 List<String> noms = new ArrayList<String>(50) {  
2     @Override  
3     public void add(int index, String élément) {  
4         super.add(index, élément);  
5         System.out.printf("Ajoute %s à l'indice  
6         %d\n", élément, index);  
7     }  
}
```

⚠ Les méthodes redéfinies ne doivent bien sûr pas être **final**

Bilan intermédiaire pour la conception des classes

- Respecter l'encapsulation des données
 - avec les méthodes d'accès, toute modification de leur représentation n'affecte pas le code utilisateur
 - plus facile de localiser et de corriger les erreurs
 - ne pas créer systématiquement des méthodes d'accès et d'altération pour tous les champs
 - Initialiser les données (et garder les objets dans un état cohérent)
 - Limiter l'utilisation de types de base dans les classes
 - remplacer les champs nombreux par des classes lorsque approprié (ex: classe `Adresse` pour `rue`, `numéro`, `codePostal`, etc.)
 - Découper les classes ayant trop de responsabilités et avoir recours aux méthodes abstraites et interfaces
 - Écrire "physiquement" les classes de manière cohérente
 - pas de "bonne solution" universelle
 - Sun recommandait de lister les champs puis les méthodes
 - autre recommandation : lister d'abord l'interface publique puis les détails de l'implémentation privée
-

Prévention et gestion des erreurs

Les exceptions

Notion d'exception

- En Java, le **traitement des erreurs** repose fortement sur la notion d'**exception** ➞
 - une méthode générant une erreur ne retourne pas une valeur particulière (qu'il faudrait interpréter comme *telle erreur*)
 - mais elle "lève" ("déclenche", "génère", "propage", "lance") une exception
 - qui déclenche la recherche d'un **gestionnaire d'exception** pouvant la traiter (un bloc de code)
 - Dans le monde objet de Java, toute exception est une sous-classe de `java.lang.Throwable`
-

Types d'exceptions

- Exceptions dites **sous contrôle**, ou *vérifiées* (**checked exception**) : liées à une cause externe au programme (ex : erreur d'entrée/sortie : `java.io.IOException`)
 - Exception dites **hors contrôle**, ou *non vérifiées* (**unchecked exception**) : dérivant de `RuntimeException`, liées à une erreur de programmation (ex : invocation d'une méthode sur une référence `null` : `NullPointerException`)
 - **Erreurs** : dérivant de `java.lang.Error`, représentation des conditions anormales ne devant en général pas être traitées (ex.: erreur de la machine virtuelle `VirtualMachineError`)
-

Propagation d'exceptions (`throws`, `throw`)

- Lorsqu'une méthode est susceptible de propager une exception sous contrôle, elle doit le signaler/déclarer dans sa signature à l'aide du mot-clé `throws`, ex.

(si elle n'en assure pas elle-même le traitement ↪)

```
public int chargementFichier(String nomFichier) throws  
FileNotFoundException
```

cas de propagation d'exception

- Une méthode invoque une méthode elle-même susceptible de propager une telle exception (et que la méthode ne traite pas directement)
 - Une méthode peut identifier une situation correspondant à une exception, et donc la créer (`new`) et la propager (`throw`)
-

② est-il possible/souhaitable qu'une méthode déclare plusieurs exceptions sous contrôle ? >

C'est au moins possible : une méthode doit indiquer la ou les **exceptions sous contrôle** (en les énumérant) .

② que doit-il se passer si une méthode cherche à propager une exception qu'elle ne déclare pas, et pourquoi ? >

L'absence de déclaration d'exception sous contrôle au niveau d'une méthode avec `throws` ou invocation d'une méthode pouvant elle-même propager une exception sous-contrôle est détectée par le compilateur.

② qu'en est-il des exceptions hors contrôle (*unchecked*), ex. `ArrayIndexOutOfBoundsException` ? >

Celles-ci n'ont pas à être signalées, mais elles peuvent être propagées.

Propagation d'exceptions et héritage

② que doit-on faire vis-à-vis des exceptions lors de la redéclaration de méthodes ?

>

⚠ Une méthode qui redéfinit la méthode d'une super-classe ne peut pas propager de nouvelles exceptions sous contrôle

- mais il est possible de lancer des exceptions sous contrôle plus spécifiques
(ex. `java.io.FileNotFoundException` à la place de `java.io.IOException`)
- il est également possible de ne plus transmettre certaines exceptions

```
1  class A {
2      public void m() throws IOException {...}
3  }
4
5  class B extends A {
6      // note : les lignes ci-dessous sont
        // incompatibles entre elles
7      @Override public void m() throws IOException
        {...} // OK
8      @Override public void m() throws
        SocketException {...} // OK
9      @Override public void m() throws
        SQLException {...} // KO
10     @Override public void m() {...} // OK
11     // ...
12 }
```


📋 Identification de situations d'exception

Si une méthode est *susceptible de rencontrer ou provoquer une exception* et *n'en assure pas le traitement*, il faut :

- trouver la classe d'exception appropriée (ex. dans les API standard) ou éventuellement créer une nouvelle classe
- créer une instance de cette exception et la "lancer" (`throw`)

🔗 Le lancement d'une exception interrompt l'exécution de la méthode, qui ne retourne donc pas de valeur (notamment, pas de "code d'erreur")

☰ Exemple : on atteint la fin d'un flux avant d'avoir lu une quantité de données attendue

```
1  String lectureDonnées(Scanner in) throws
   EOFException {
2      // ...
3      if (!in.hasNext()) // fin de fichier atteinte
        (End Of File)
4          if (tailleLue < tailleAttendue)
5              throw new EOFException("Attendu : " +
tailleAttendue + " | lu : " + tailleLue);
6          // dans ce cas, la méthode est quittée
           immédiatement
7      // ...
8  }
```

Définition d'exceptions

Nouveaux types d'exceptions

Pour permettre un traitement fin des erreurs, il peut être utile de définir de **nouveaux types d'exceptions** :

- cela se fait par **dérivation** d'une classe d'exception, à partir de `java.lang.Exception` ou d'un type d'exception plus précis (ex.: `java.io.IOException`)
- il est d'usage d'implémenter au moins deux constructeurs
 - un sans paramètres, et un avec un paramètre définissant un message décrivant l'exception (tel que défini par `java.lang.Throwable`)
 - la méthode `String getMessage()` de `Throwable` permet d'avoir accès à cette description

```
1  class FormatDeFichierException extends IOException
   {
2      public FormatDeFichierException() {}
3      public FormatDeFichierException(String
        description)
4          { super(description); }
5  }
```

Une fois un tel type d'exception déclaré et rendu accessible à une classe, ses méthodes peuvent en propager des instances :

```
throw new FormatDeFichierException("Attendu : " + ...)
```

Survenue d'exceptions hors contrôle (*unchecked*)

🔗 Lorsqu'une exécution rencontre une exception hors contrôle non capturée, le programme se termine immédiatement et une trace de la pile d'exécution est affichée avec l'exception correspondante

```
1  public class A {  
2  
3      public void m1() {  
4          m2();  
5      }  
6  
7      public void m2() {  
8          A a = null;  
9          a.m3();  
10     }  
11  
12     public static void main(String[] args) {  
13         A a = new A();  
14         a.m1();  
15     }  
16 }
```

```
1  Exception in thread "main"  
   java.lang.NullPointerException  
2      at A.m2(A.java:9)  
3      at A.m1(A.java:4)  
4      at A.main(A.java:14)
```

Capture d'exceptions sous contrôle (checked)

🔗 Erreur à la compilation si rien n'est fait lors de l'appel d'une méthode déclarant une exception sous contrôle

```
1  public class A {  
2  
3      public void m() throws java.io.IOException {  
4          // ...  
5      }  
6  
7      public static void main(String[] args) {  
8          A a = new A();  
9          a.m();  
10     }  
11 }
```

```
1  A.java:9: error: unreported exception IOException;  
2      must be caught or declared to be thrown  
3      a.m();          ^  
4  1 error
```

Capture d'exceptions

- Lorsqu'on invoque une méthode déclarant une **exception sous contrôle** (i.e. susceptible de la propager), il peut être nécessaire de la "capturer" à l'aide d'un bloc `try` / `catch` (**gestionnaire d'exceptions**)

```
1 try {  
2     // code susceptible de lancer une exception de  
   type TypeException  
3 }  
4 catch (TypeException e) {  
5     // traitement de l'exception  
6 }
```

Gestionnaires d'exception

- le code présent après le (possible) lancement de l'exception dans le bloc `try` ne sera pas exécuté
- possibilité d'avoir plusieurs blocs `catch` correspondant à différents types d'exceptions
- possibilité de factoriser plusieurs types d'exceptions non liés pour un même gestionnaire (*multicatch*) :

```
catch (TypeEx1 | TypeEx2 e)
```

🔗 doit-on capturer localement toutes les exceptions ? >

Il est possible de capturer localement certaines exceptions (`try` / `catch`) et d'en propager d'autres (`throw` / `throws`) en fonction de la responsabilité du code et des possibilités de traitement.

Capture d'exceptions multiples

- Un même bloc de code est susceptible d'être confronté à plusieurs types d'exceptions
 - (attention à éviter les longs blocs avec trop de responsabilités)
 - on peut intercepter de façon fine chaque type d'exception
 - le premier gestionnaire d'exception compatible est exécuté
 - attention aux relations de types entre les différentes exceptions
 - il reste possible de ne pas intercepter certaines exceptions, lesquelles doivent alors être propagées par la méthode incluant le bloc (déclaration avec `throws`)

≡ Exemple de captures / propagations

```
1  <signature_méthode> throws Exception1 {
2      // ...
3      try {
4          // code susceptible de lancer des
           exceptions
5      }
6      catch (Exception2 | Exception3 e) {
7          // multi-catch
8          // traitement des exceptions Exception2 ou
           Exception3
9      }
10     catch (Exception4 e) {
11         // traitement des exceptions Exception4
12         // (sauf si... Exception4 est un sous-type
           de Exception2
13         // ou de Exception3)
```

```
14     }
15     // les exceptions de type Exception1 ne sont
    pas
16     // traitées localement
17     // ...
18 }
```

Traitement partiel d'exception

- Il est possible d'intercepter une exception et de la "repropager"
- Il est également possible de propager une nouvelle exception suite au traitement d'une exception
 - cela permet de transmettre une exception adaptée pour le code appelant, en particulier lorsque la première exception correspond à un type d'erreur très interne et spécifique au code
 - la méthode `initCause` de `Throwable` permet de lier les exceptions

≡ Exemple : propagation d'une nouvelle exception liée

```
1  try {
2      // code d'accès à une base de données
3      // ...
4  }
5  catch (SQLException e) {
6      // pas nécessairement pertinent pour le code
        appelant
7      Throwable se = new ServletException("Erreur
        liée à la base de données");
8      se.initCause(e); // renseigne sur l'exception
        d'origine
9      throw se; // propagation de la nouvelle
        exception
10 }
```

② est-ce un problème que la survenue d'une exception entraîne le départ immédiat du bloc `try` ?

Garanties d'exécution en cas d'exception : la clause `finally`

- Il peut être utile de **garantir que certaines instructions soient exécutées** quelle que soit l'exécution d'un bloc (normale ou interrompue par la survenue d'une exception)
 - ceci est possible avec la clause `finally`, associée à un bloc `try` et exécutée juste avant la sortie du bloc ou de la méthode en cas de propagation d'exception
 - permet en particulier de libérer proprement des ressources locales

🔗 Exécution du bloc `finally`

Ne nécessite pas qu'une exception survienne : garantit simplement qu'un code sera toujours exécuté

☰ Exemple de traitement par `finally`

```
1  InputStream in = ... ; // création d'une ressource
   locale
2  try {
3      // code susceptible de générer des exceptions,
4      // qui seraient propagées ici
5  }
6  finally {
7      // le traitement d'une exception avec catch
8      // n'est pas nécessaire
9      in.close(); // garantit que la ressource
   locale
10                      // sera bien libérée lorsque
11                      // la méthode sera quittée
12 }
```

② que se passe-t-il si une nouvelle exception est levée dans le bloc `finally` ? >

Si le bloc `finally` génère lui-même une exception et la propage, une éventuelle exception initiale est masquée par cette nouvelle exception

- contraire à l'esprit de la gestion d'erreur
- il faut donc éviter de propager des exceptions dans les opérations de "nettoyage"

② que se passe-t-il si un bloc `finally` contient une instruction `return` ? >

Une conséquence de la spécification de `finally` est qu'une instruction `return` y devient prioritaire sur celle présente dans le bloc `try` associé

```
1  public int m(int p) {  
2      try {  
3          int r = p * p;  
4          return r;  
5      }  
6      finally {  
7          if (p == 2) return 0; // valeur utilisée !  
8      }  
9  }
```

② serait-il envisageable qu'on oublie (ou qu'on trouve fastidieux) de libérer des ressources dans des blocs `finally` ?

Blocs `try` associés à des ressources : *try-with-resource* (☞ 7)

Alternative à l'écriture `try...finally` pour la libération des ressources : *try-with-resource* (☞ 7)

```
1  void lecture() throws IOException {
2      try(
3          FileInputStream in = new
4          FileInputStream("in.txt");
5          BufferedInputStream inBuffer = new
6          BufferedInputStream(in)
7      ) {
8          int donnees = inBuffer.read();
9          while(donnees != -1) {
10             System.out.print((char) donnees);
11             donnees = inBuffer.read();
12         }
13     }
```

❓ comment l'implémentation est-elle possible ? >

Les "ressources" doivent implémenter l'interface `java.lang.AutoCloseable` (ex. ~ `FileInputStream`).

✍ La première exception déclenchée depuis le bloc `try` est cette fois prioritaire sur celles qui pourraient être déclenchées en libérant les ressources

Exceptions : bonnes pratiques

⚠ coût du traitement par exceptions

Les exceptions n'ont pas pour rôle de se substituer à certains tests simples

- le mécanisme des exceptions est notamment beaucoup plus lourd en temps de traitement
- ex : dépiler 10 millions de fois une pile vide

```
1  if (!s.empty()) s.pop; // 646ms
2
3  // Vs
4
5  try { // 21,739s
6      s.pop();
7  } catch (EmptyStackException e) { }
```

⚠ responsabilité du traitement des exceptions

Il ne faut pas vouloir traiter les exceptions localement à tout prix

- dans de nombreux cas, la propagation d'une erreur sera préférable, car le code de niveau supérieur sera plus à même de gérer l'erreur au niveau de l'application globale
-

② pourquoi le recours aux exceptions sous contrôle (*checked*) est très débattu ?

Exceptions : exemple

```
1  class EntierNaturel {
2
3      private int n;
4      public EntierNaturel(int n) throws ErrConst {
5          if (n < 0) throw new ErrConst(n);
6          this.n = n;
7      }
8
9      public static EntierNaturel somme (EntierNaturel
n1, EntierNaturel n2)
10         throws ErrSom, ErrConst {
11         int op1 = n1.n, op2 = n2.n;
12         long s = op1 + op2;
13         if (s > Integer.MAX_VALUE) throw new ErrSom(op1,
op2);
14         return new EntierNaturel(op1 + op2);
15     }
16     public static EntierNaturel diff (EntierNaturel n1,
EntierNaturel n2)
17         throws ErrDiff, ErrConst {
18         int op1 = n1.n, op2 = n2.n;
19         int d = op1 - op2;
20         if (d < 0) throw new ErrDiff(op1, op2);
21         return new EntierNaturel(d);
22     }
23     public static EntierNaturel prod (EntierNaturel n1,
EntierNaturel n2)
24         throws ErrProd, ErrConst {
25         int op1 = n1.n, op2 = n2.n;
26         long p = (long)op1 * (long)op2;
27         if (p > Integer.MAX_VALUE) throw new ErrProd(op1,
op2);
28         return new EntierNaturel((int)p);
29     }
```



```
30     // ...
31 }
32
33
34 class ErrNat extends Exception {}
35
36 class ErrConst extends ErrNat {
37     public int n;
38     public ErrConst (int n) { this.n = n; }
39 }
40
41 class ErrOp extends ErrNat {
42     public int n1, n2;
43     public ErrOp(int n1, int n2) {
44         this.n1 = n1; this.n2 = n2;
45     }
46 }
47
48 class ErrSom extends ErrOp {
49     public ErrSom(int n1, int n2) {
50         super(n1, n2);
51     }
52 }
53
54 class ErrDiff extends ErrOp {
55     public ErrDiff(int n1, int n2) {
56         super(n1, n2);
57     }
58 }
59
60 class ErrProd extends ErrOp {
61     // ...
62 }
63
64 // ...
65 try {
```

```
66     EntierNaturel n1 = new EntierNaturel(50000);
67     EntierNaturel n2 = new EntierNaturel(45000);
68
69     EntierNaturel d = EntierNaturel.diff(n1, n2);
70     EntierNaturel s = EntierNaturel.somme(n1, n2);
71     EntierNaturel p = EntierNaturel.prod(n1, n2);
72 }
73 catch (ErrConst e) {
74     System.err.println("erreur construction entier avec
75 : " + e.n);
76 }
77 catch (ErrDiff e) {
78     System.err.println("erreur différence avec " + e.n1
79 + " et " + e.n2);
80 }
81 catch (ErrSom e) {
82     System.err.println("erreur somme avec " + e.n1 + "
83 et " + e.n2);
84 }
85 catch (ErrProd e) {
86     System.err.println("erreur produit avec " + e.n1 +
87 " et " + e.n2);
88 }
89 // ...
```

Les assertions

- Lors de la mise au point et du test de programmes, il peut être utile d'effectuer certains tests qui ne seront pas effectués lors d'une utilisation en production
- Java propose le mécanisme des **assertions** ([§ 1.4](#))
 - on explicite une condition qui devrait être vraie à un endroit du code
 - si la condition n'est pas vérifiée, une exception de type `AssertionError` est levée
 - il faut explicitement activer les assertions pour la JVM

```
$ java -enableassertions Classe
```

🔗 exceptions vs. assertions

Contrairement aux exceptions, **la cause d'une assertion non vérifiée n'a pas vocation à être traitée** pour pouvoir poursuivre l'exécution du programme : il s'agit de corriger une erreur de programmation (ex.: prendre en compte un cas non valide).

⚙️ **exemple : on souhaite s'assurer que la valeur d'une variable entière n'est pas négative**

```
assert variable >= 0 : "variable est négative (!)";
```

Assertions et tests unitaires

- Pratique importante de **développement guidé par les tests** (*test-driven development*)
- Frameworks additionnels pour les **tests unitaires** 🐞 en Java
 - ex. JUnit (très utilisé), qui repose sur l'annotation de méthodes (`@Test`), l'inspection du code et les assertions (classe `org.junit.Assert`), ex.

```
1 public class TestPersonne {
2     @Test
3     public void testConcatNom() {
4         Personne p = new Personne("Victor", "Hugo");
5         String concat = p.getNomComple();
6         assertEquals("Concaténation nom", "Victor
    Hugo", concat);
7     }
8 }
```

- Exécution *via* une classe du framework, fonctions spéciales dans les IDEs

```
1 $ javac -cp . TestPersonne.java
2 $ java -cp junit4.x.x.jar junit.textui.TestRunner \
3     projet.TestPersonne
4 Time: 0.170
5 OK (1 tests)
```

Le framework JUnit

- Utilisation d'assertions définies comme méthodes de `org.junit.Assert`
 - `assertEquals`, `assertTrue/False`, `assert(Not)Null`, `assert(Not)Same`, `assertArrayEquals`
 - indications des assertions qui échouent
 - Définition de **cas de test** (*test cases*) avec `junit.framework.TestCase`
 - annotation de méthodes exécutées :
 - avant et après les méthodes de test (`@Before`, `After`)
 - avant et après toutes les méthodes (`@BeforeClass`, `@AfterClass`)
 - méthodes ignorées (`@Ignore`)
 - pas de méthode `main` nécessaire, les tests sont indépendants
 - Définition de **suites de tests** (*test suites*) qui exécutent un certain nombre de cas de test
-

Le framework JUnit : exemple de TestCase

```
1  import junit.framework.TestCase;
2  import org.junit.*;
3
4  public class TestPersonne extends TestCase {
5      @BeforeClass public static void setupClasse()
6          throws Exception {
7          // ...
8      }
9      @Before public void setupTest() throws Exception
10     {
11         // ...
12     }
13     @Test public void monTest1() {
14         // ...
15     }
16     @Test public void monTest2() {
17         // ...
18     }
19     @After public void tearDownTest() throws
20     Exception {
21         // ...
22     }
23     @AfterClass public static void tearDownClasse()
24     throws Exception {
25         // ...
26     }
27 }
```

La consignation en Java

- Java (☞ 1.4) permet une approche cohérente de la **journalisation des événements d'intérêt** survenant lors de l'exécution des programmes : la **consignation** (*logging*) ☞
 - différents gestionnaires peuvent traiter les entrées
 - les formats de sortie peuvent être configurés finement et être traités par un code de **localisation** ☞
 - chaque entrée est affectée à un **niveau de consignation** et chaque gestionnaire traite un niveau minimum
 - ces niveaux sont définis par des champs statiques de `java.util.logging.Level` : `SEVERE`, `WARNING`, `INFO`, `CONFIG`, `FINE`, `FINER`, `FINEST`
- La classe `java.util.logging.Logger` dispose d'un champ statique permettant d'utiliser simplement la consignation :

```
Logger.global.log(Level.FINER, "Test machin réussi");
```

 - on peut facilement suspendre la consignation :

```
Logger.global.setLevel(Level.OFF);
```
 - il est possible de définir des gestionnaires configurables

☺ qu'est-ce que la localisation (*internationalization*) ?

≡ Exemples de consignations

```
1  if (condition) {
2      IOException exception = new IOException("...");
3      logger.throwing("fr.polytech.ia.Reader",
4                      "read", exception);
5      throw exception;
6  }
```

```
1  try {
2      ...
3  }
4  catch (IOException exception) {
5
6      Logger.getLogger("fr.polytech.ir.ImageRetrieval")
7          .log(Level.WARNING, "Chargement d'image",
8              exception);
9  }
```

Programmation générique et collections

② ce type d'écriture présente-t-il des défauts ?

```
1  java.util.List ingés3 = new java.util.ArrayList();  
2  // ...  
3  Etudiant unEtudiant =  
    (Etudiant)ingés3.get(indice);
```

② ce type d'écriture présente-t-il des défauts ?

```
1  class PaireEtudiants {  
2      private Etudiant étudiant1;  
3      private Etudiant étudiant2;  
4      // ...  
5  }
```

📄 Détection d'erreurs à la compilation

Dans certains cas, l'écriture de programmes en Java peut amener à une gestion relativement laborieuse (et risquée) pour qu'un même code puisse manipuler des objets de types différents

- recours au transtypage d'objets (ex. lors du parcours de collections d'objets, méthodes telles que `compareTo(Object autre)`)
- l'échec d'un transtypage vers une sous-classe de `java.lang.Object` ne peut être détecté qu'à l'exécution

```
1  java.util.List ingés3 = new java.util.ArrayList();
2  // ... initialisation de la liste ingés3 ...
3  // rien n'impose que ce soit effectivement un
   Etudiant
4  Etudiant unEtudiant =
   (Etudiant) ingés3.get(indice);
```

🔗 Introduction d'éléments de programmation générique 🔗

par le biais de **types et méthodes paramétrés** 🔗 (📖 5.0)

```
List<Etudiant> ingés3 = new ArrayList<Etudiant>();
```

📖 en Java, les espaces autour des chevrons (< >) ne sont pas significatifs.

Types paramétrés

📋 Les types paramétrés permettent :

- une **sécurisation des programmes** : certaines erreurs sont détectées à la compilation et non plus à l'exécution
- une **amélioration de leur lisibilité** : le type des objets manipulés est apparent

📋 Définitions de types paramétrés

- de nombreuses classes et interfaces des API standard ont une version paramétrée, ex.
la classe `java.util.ArrayList<E>`
l'interface `java.lang.Comparable<T>`
- qui cohabitent avec des versions non paramétrées pour des raisons de compatibilité et d'implémentation, ex.
la classe `java.util.ArrayList`
l'interface `java.lang.Comparable`
- possibilité (évidemment) de définir ses propres types paramétrés

② pourquoi cette cohabitation entre types paramétrés et leurs correspondants non paramétrés ?

🔗 Il n'est plus nécessaire de transtyper les références avec les types paramétrés

```
1 List< Etudiant > ingés3 = new ArrayList< Etudiant  
  >();  
2 // ... initialisation de la liste ingés3 ...  
3 // OK : c'est nécessairement un Etudiant, le  
  compilateur le sait  
4 // pas de ClassCastException à l'exécution  
5 Etudiant étudiant = ingés3.get(0);
```

⚠ La manipulation de types incompatibles provoquera une erreur à la compilation

```
1 List< Etudiant > ingés3 = new ArrayList< Etudiant  
  >();  
2 // ...  
3 ingés3.add(new Enseignant("Platon"));
```

```
1 XXX.java:10: error: incompatible types: Enseignant  
2 cannot be converted to Etudiant  
3     ingés3.add(new Enseignant("Platon"));
```

Définition de types génériques

- La définition d'un **type générique** fait apparaître une ou plusieurs **variables de type**
 - ex. `classe<[VariablesType] +>`
 - l'API standard utilise les identificateurs **T**, **U** et **S** pour des éléments de type quelconque, **E** pour des éléments de collections, et **K** et **V** pour les clés et valeurs d'une table d'association

≡ Exemple : paires d'éléments de même type

```
1 public class Paire< T > {  
2     private T premier, second;  
3     public Paire(T p, T s) { premier = p; second =  
        s; }  
4     public T getPremier() { return premier; }  
5     public void setPremier(T valeur) { premier =  
        valeur; }  
6 }
```

```
1 Paire< String> paireChaînes = new Paire< String >  
    ("ping", "pong");  
2 Paire< String > paireChaînes = new Paire< >  
    ("ping", "pong");  
3 Paire< Integer > paireEntiers = new Paire< Integer  
    >(1, 2);  
4 Paire< Integer > paireEntiers = new Paire< >(1,  
    2);  
5 var paireDouble = new Paire< Double >(d1, d2); //  
    (Java10)
```

Définition de méthodes génériques

- Il est possible de définir des **méthodes génériques**
 - il n'est pas nécessaire qu'une classe soit générique pour qu'elle possède des méthodes génériques
 - les variables de type doivent être insérées entre les modificateurs et le type de retour

```
1  class TestMéthodeGénérique {
2      public static <T> T getElémentDuMilieu(T[]
    tableau) {
3          return tableau[tableau.length / 2]; // ...
4      }
5  }
```

- L'appel peut se faire en spécifiant explicitement les types pour la méthode générique
 - le compilateur saura toutefois (dans la plupart des cas) inférer les types pour la méthode

```
1  String[] tableau = { "préfixe", "infixe", "suffixe"
    };
2  String s = TestMéthodeGénérique.
    <String>getElémentDuMilieu(tableau);
3  String s =
    TestMéthodeGénérique.getElémentDuMilieu(tableau); //
    OK
```

≡ Exemple: tirage d'une valeur au hasard dans un tableau

```
1  public class Util {
2      public static < T > T hasard (T[] valeurs) {
3          if (valeurs == null || valeurs.length ==
4              0)
5              return null;
6          int ind = (int) (valeurs.length *
7              Math.random());
8          return valeurs[ind];
9      }
10
11     public static void main(String[] args) {
12         Integer[] tabInt = {1, 2, 3, 4, 5};
13         String[] tabString = {"un", "deux",
14             "trois", "quatre"};
15         System.out.println(hasard(tabInt) + ", " +
16             hasard(tabString));
17     }
18 }
```

Contraindre les variables de type

- Pour une classe ou une méthode générique, il est possible de préciser qu'une variable de type `T` doit être d'un type particulier (**type limitant** (*bounding type*))
 - `< T extends TypeLimitant >`
 - `< T extends TypeLimitant1 & TypeLimitant2 >`
- Cela permet de garantir que tous les objets disposent de certaines propriétés

❓ pourquoi peut-on avoir plusieurs interfaces comme types limitants, mais au plus une seule classe (mentionnée alors en premier) ?

≡ Extraction d'une valeur minimale d'un tableau

```
1  public static < T extends Comparable< T >> T
   min(T[] tableau) {
2      if (tableau == null || tableau.length == 0)
3          return null;
4      T plusPetit = tableau[0];
5      for (int indice = 1 ; indice < tableau.length
   ; indice++)
6          // les objets de tableau disposent
   nécessairement
7          // de cette méthode
8          if (plusPetit.compareTo(tableau[indice]) >
   0)
9              plusPetit = tableau[indice];
10     return plusPetit;
11 }
```

≡ Limitation des paramètres de types d'une méthode

```
1  class Point implements Comparable< Point > {
2      private int x, y;
3      //...
4      public int compareTo(Point p) {
5          int norme1 = x * x + y * y, norme2 = p.x *
p.x + p.y * p.y;
6          if (norme1 == norme2) return 0;
7          if (norme1 > norme2) return 1;
8          return -1;
9      }
10 }
11
12 class Util {
13     // ...
14     static < T extends Comparable< T > > T max
(T[] valeurs) {
15         if (valeurs == null || valeurs.length ==
0) return null;
16         T max = valeurs[0];
17         for (T valeur : valeurs)
18             if (valeur.compareTo(max) > 0)
19                 max = valeur;
20         return max;
21     }
22 }
23
24 // ...
25 Point[] points = { new Point(0, 1), new Point(2,
3), ...};
26 Point maxPoint = Util.max(points);
```

Types bruts de la machine virtuelle

📋 La machine virtuelle ne voit que des classes ordinaires

- les types génériques sont donc transformés en types bruts
- les variables de type sont supprimées (**effacement des types** (*type erasure*)) et remplacées par leur type limitant (`Object` le cas échéant)
- la machine virtuelle réalise les transtypages nécessaires

☰ exemples d'effacements de types

- `List< ? extends Number > → List`
- `< T extends Comparable< T > > T max(T o1, o2)`
→ `Comparable max(Comparable o1, Comparable o2)`

✍ Quelques conséquences

- les types primitifs ne peuvent pas être utilisés comme paramètres de type
 - les tableaux de types avec paramètres sont impossibles
 - il n'est pas possible d'instancier (directement) des variables de type (cela pourrait revenir à instancier `Object`)
-

Types génériques et héritage

- Les classes génériques peuvent (bien sûr) étendre/implémenter d'autres classes/interfaces génériques
 - ex : `ArrayList< E >` implémente `List< E >`

⚠ pas de relation entre `Type< T1 >` et `Type< T2 >` pour tous `T1` et `T2` différents

ex. : pas de relation d'héritage entre ex. `Paire< Personne >` et `Paire< Etudiant >`

```
1  Paire< Etudiant > binôme = new Paire<>();
2  Paire< Personne > couple = binôme; // interdit
```

✍ différence notable avec les tableaux

protégés par une vérification de type à l'exécution :

`ArrayStoreException`

```
1  Etudiant[] étudiants = {julesVerne,
    arthurConanDoyle, victorHugo};
2  Personne[] gens = étudiants; // permis
```

≡ Exemple de dérivation de classe générique

```
1  class Couple< T > {
2      private T x, y;
3      public Couple(T premier, T second) { x =
premier; y = second; }
4      // ...
5  }
```

≡ un couple nommé ? >

```
1  class CoupleNommé< T > extends Couple< T > {
2      private String nom;
3      public CoupleNommé(T premier, T second,
String nom) {
4          super(premier, second); this.nom = nom;
5      }
6  }
```

≡ un point nommé ? >

```
1  class PointNommé extends CoupleNommé< Integer
> {
2      public PointNommé(Integer premier, Integer
second, String nom) {
3          super(premier, second, nom);
4      }
5  }
6
7  // ...
8  Couple< Double > cd1 = new Couple< Double >
(1.0, 2.0);
```

```
9  Couple< Double > cd2 = new Couple< Double >
    (3.0, 4.0);
10 CoupleNommé< String > cns =
11     new CoupleNommé< String >("astérix",
    "obélix", "les invincibles gaulois");
12 CoupleNommé< Couple< Double >> cnd =
13     new CoupleNommé< Couple< Double >> (cd1,
    cd2, "un couple de couples");
```

Jockers de types génériques

- Il est possible de contraindre l'utilisation de classes/méthodes paramétrées à l'aide de **types jockers** (*wildcards*)
- limite de sous-type : `? extends TypeDeBase`
 - n'importe quel sous-type de `TypeDeBase` (inclus) : **accès en lecture**

```
1 void prendreNouvelles(Collection<? extends Etudiant>  
   etudiants) {...}
```

- limite de super-type : `? super TypeDeBase`
 - n'importe quel super-type de `TypeDeBase` (inclus) : **accès en écriture**

```
1 void ajouteEtudiantPoPS(Collection<? super  
   EtudiantPoPS> etudiants) {...}
```

Collections d'objets

- Les API de Java apportent un ensemble d'interfaces et classes pour représenter des **collections d'objets**
 - stockage, transmission, manipulation et recherche
- Distinction entre les *interfaces* et les *implémentations*

≡ collection de type *file*

- correspond à une certaine interface

```
1 interface Queue< E > {  
2     void add(E element);  
3     E remove();  
4     int size();  
5 }
```

- peut être implémentée de diverses manières, par exemple comme une liste chaînée

```
1 class FileListeChaînée< E > implements Queue< E >  
2 {  
3     private E tête, queue;  
4     FileListeChaînée() { ... }  
5     @Override public void add(E element) { ... }  
6     @Override public E remove() { ... }  
7     @Override public int size() { ... }  
8 }
```

Interfaces de collections

- On peut faire référence aux **collections** par leur type d'interface
 - cela permet de **changer d'implémentation de manière transparente**

```
1 Queue<Etudiant> poseursDeQuestions = new
  FileListeChaînée<Etudiant>();
2 poseursDeQuestions.add(new Etudiant("laurel"));
```

- Le choix de l'**implémentation** se fait par des considérations propres aux besoins de l'application
 - ex : on peut parfois préférer des collections bornées d'accès plus rapide, parfois des collections non bornées d'accès plus lent, des collections modifiables ou non

🔗 Interface fondamentale : `java.util.Collection< E >`

```
1 public interface Collection< E > {
2     // renvoie true si l'ajout a été effectivement
    fait
3     boolean add(E element);
4     // renvoie un objet permettant de parcourir
5     // les éléments de la collection
6     Iterator< E > iterator();
7 }
```

Itérateurs de collections

- On peut parcourir itérativement les éléments d'une collection à l'aide d'objets implémentant `java.util.Iterator< E >`

```
1 public interface Iterator<E> {  
2     E next() throws NoSuchElementException;  
3     boolean hasNext();  
4     void remove() throws  
        UnsupportedOperationException;  
5 }
```

≡ Parcours effectif

```
1 Collection< Etudiant > ingés3 = ... ;  
2 Iterator< Etudiant > itérateur =  
    ingés3.iterator();  
3 while(itérateur.hasNext()) {  
4     Etudiant étudiant = itérateur.next();  
5     // ...  
6 }
```

nouvelle utilisation de l'instruction `for` (5.0)

(conversions par le compilateur utilisant un itérateur)

```
1 for (Etudiant étudiant : ingés3) { ... }
```

≡ Exemple: maintenir une liste triée

```
1  class Promotion {
2
3      private List< Etudiant > etudiants;
4
5      public Promotion() {
6          etudiants = new ArrayList< Etudiant >();
7      }
8
9      public void ajoute(Etudiant étudiant) {
10         ListIterator< Etudiant > it =
etudiants.listIterator();
11         // public interface ListIterator< E >
extends Iterator< E >
12         // An iterator for lists that allows the
programmer to traverse
13         // the list in either direction, modify
the list during iteration,
14         // and obtain the iterator's current
position in the list
15         boolean trouvé = false;
16         while ( it.hasNext() && !trouvé )
17             if ( it.next().compareTo(étudiant) > 0
)
18                 trouvé = true;
19                 if (trouvé) it.previous();
20                 it.add(étudiant);
21     }
22
23     // ...
24 }
```

Méthodes utilitaires génériques

- La bibliothèque standard propose des méthodes (`java.util.AbstractCollection< E >`) travaillant sur n'importe quel type de collection, notamment :
 - accès à une collection, ex.

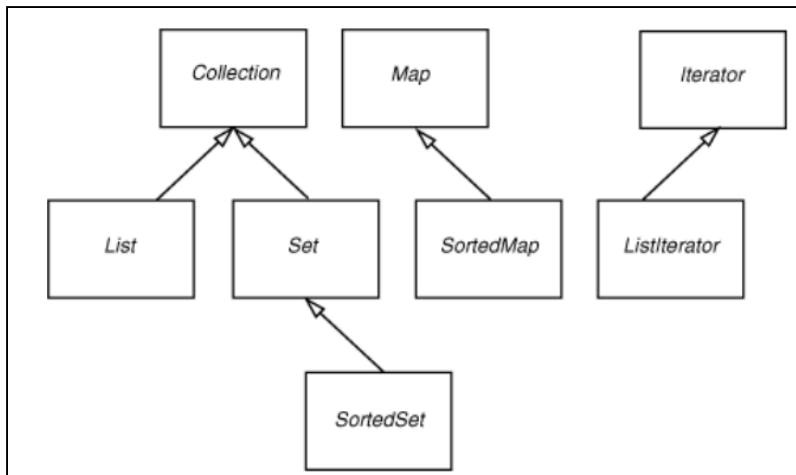
```
1 // retourne le nombre d'éléments
2 int size();
3
4 // indique si une collection est vide
5 boolean isEmpty();
6
7 // indique si la collection contient un objet
8 boolean contains(Object objet);
9
10 // indique si tous les éléments d'une autre
    collection
11 // sont contenus
12 boolean containsAll(Collection< ? > autre);
13
14 // renvoie un tableau contenant les objets de la
    collection
15 Object[] toArray();
```

- altération d'une collection, ex.

```
1 // ajoute un élément et retourne true si la
  collection
2 // a été modifiée
3 boolean add(Object objet);
4
5 // ajoute un ensemble d'éléments
6 boolean addAll(Collection< ? extends E > autre);
7
8 // renvoie true si l'objet a été trouvé
9 boolean remove(Object obj);
10
11 // enlève tous les éléments d'une autre collection
12 // et retourne true si la collection a été modifiée
13 boolean removeAll(Collection< ? > autre);
14
15 // supprime tous les éléments
16 void clear();
```

Interfaces du cadre des collections

- Cadre complet pour les collections, avec notamment :
 - `List` : une collection ordonnée, avec des méthodes permettant d'accéder aux éléments par leur indice
 - `Set` : une collection n'autorisant pas les valeurs dupliquées, dont l'ordre n'est pas toujours important
 - `Map` : une carte associant des clés et des valeurs
 - `SortedSet` / `SortedMap`



Collections concrètes

- La bibliothèque standard propose de nombreuses collections concrètes, notamment :
 - `ArrayList` : séquence indexée qui grandit et se réduit de manière dynamique
 - `LinkedList` : séquence ordonnée qui permet des insertions et des suppressions efficaces n'importe où
 - `HashSet` : collection non ordonnée qui interdit les duplications
 - `TreeSet` : ensemble trié
 - `HashMap` : structure de données qui stocke des associations clé/valeur
 - `TreeMap` : concordance dans laquelle les clés sont triées
-

Vues sur collections

- Les **vues** sur collections sont des objets légers qui implémentent une interface de collection, mais ne stockent pas directement d'éléments
 - toute modification (autorisée) de la collection modifie la vue (et *vice versa*)
- En général, les vues ne supportent pas toutes les opérations de leur interface
 - ex. dans `Map< K, V >`, la méthode `keySet()` retourne une vue sur l'ensemble des clés de l'objet (de type `Set< K >`)
 - seules les opérations de type *remove* sont supportées sur le *set*, pas celles de type *add*

❓ voudrait-on des vues non altérables, et pourquoi ? >

Possibilité d'obtenir des **vues non altérables**

- invoquer une méthode d'altération sur elles lève une exception (`UnsupportedOperationException`)

```
1  public class EtudiantPolytech {
2      private List< AssociationBDE > assos;
3
4      public List< AssociationBDE > getAssos() {
5          return
6              Collections.unmodifiableList(assos);
7      }
```

Algorithmes et collections

- La bibliothèque standard propose des implémentations efficaces pour de nombreux algorithmes sur des collections
 - Exemple : tri de `List`
 - si les éléments d'une liste implémentent l'interface

`Comparable< T >`

```
1 List< String > mots = new LinkedList< String >();
2 // lecture des mots et remplissage de la liste
3 // ...
4 Collections.sort(mots);
```

- possibilité de spécifier un objet réalisant les comparaisons : interface `java.util.Comparator< T >`

```
1 Comparator<Etudiant>
  compareteurEtudiantParSympathie
2     = new Comparator<Etudiant>() {
3         public int compare(Etudiant a, Etudiant
4             b) {
5             return b.getIndiceSympathie() -
6                 a.getIndiceSympathie();
7         }
8     }
9 // ...
10 Collections.sort(ingés3,
11     compareteurEtudiantParSympathie);
```

Exemple : utilisation de `HashMap< K, V >`

```
1  java.util.Map<String, Etudiant> ingés3
2      = new java.util.HashMap<String, Etudiant>();
3  ingés3.put("albator", new Etudiant("Sherlock",
4      "Holmes"));
5
6  // suppression si la clé était présente
7  ingés3.remove("tintin");
8
9  // remplacement d'une valeur associée à une clé
10 ingés3.put("albator", new Etudiant("John", "Watson"))
11
12 // recherche d'une entrée
13 if (ingés3.containsKey("caliméro"))
14
15     System.out.println(ingés3.get("caliméro").toString())
16
17 // parcours de l'ensemble des entrées
18 for (Map.Entry<String, Etudiant> entrée :
19     ingés3.entrySet()) {
20     String pseudo = entrée.getKey() ;
21     Etudiant étudiant = entrée.getValue() ;
22     System.out.println(pseudo + " : " +
23         étudiant.toString());
24 }
```

La méthode `hashCode()` de `Object`

- Renvoie un code de *hashage* dérivé d'un objet
 - `hashCode` et `equals` doivent être compatibles :
si `a.equals(b)` est vrai, alors `a.hashCode()` doit avoir même valeur que `b.hashCode()`
 - nécessaire pour les objets pouvant être insérés dans une table de *hashage* (ex. `HashSet`)

🔗 contraintes

- objets égaux $\Rightarrow$ codes de hashage identiques
- objets distincts $\Rightarrow$ probabilité codes différents forte

☰ exemple pour `java.lang.String`

```
1  int hash = 0;
2  for (int i = 0; i < length(); i++) hash = 31 *
    hash + charAt(i);
```

☰ exemple pour une nouvelle classe (assistance par l'IDE)

```
1  public class Employé {
2      // ...
3      public int hashCode() {
4          return 7 * nom.hashCode()
5              + 11 * new Double(salaire).hashCode()
6              + 13 * jourEmbauche.hashCode();
7      }
8  }
```


Exemple : utilisation de `TreeMap`

☞ extrait de la JavaDoc

"The map is sorted according to the natural ordering of its keys, or by a Comparator provided at map creation time, depending on which constructor is used. (...) This implementation provides guaranteed $\log(n)$ time cost for the containsKey, get, put and remove operations."

```
1  import java.util.*;
2  class Livre {
3      private Map< String, Set< Integer >> index; //
    ensemble de pages
4      public Livre() {
5          // ...
6          index = new TreeMap< String, TreeSet< Integer
    >>();
7      }
8
9      public void ajouter(String entrée, int
    numéroPage) {
10         TreeSet< Integer > numérosExistants =
    index.get(entrée);
11         if (numérosExistants == null) {
12             TreeSet< Integer > ensemble = new
    TreeSet< Integer >();
13             ensemble.add(numéro);
14             index.put(entrée, ensemble);
15         }
16         else {
17             numérosExistants.add(numéro);
18         }
19
20     public void affiche() {
```

```
21         Set < Map.Entry < String, TreeSet< Integer >>>
           élémentsIndex =
22             index.entrySet();
23         for (Map.Entry < String, TreeSet< Integer >>
           élémentCourant : élémentsIndex) {
24             String entréeCourante =
           élémentCourant.getKey();
25             TreeSet< Integer > numéros =
           élémentCourant.getValue();
26             System.out.print(entréeCourante + " : ");
27             for (int numéro : numéros)
28                 System.out.print(numéro + " ");
29             System.out.println();
30         }
31
32         // ...
33     }
```